🧠 AI Evolution Program

Parte 05 — Lenguaje, visión, audio e IA multimodal

Estudia cómo los sistemas perciben y combinan texto, imágenes, documentos, voz, audio y señales temporales.

1. 061 — Clasificación y representación visual
2. 062 — Detección, segmentación y pose
3. 063 — OCR y comprensión de documentos
4. 064 — Tokenización y representación del lenguaje
5. 065 — Clasificación, extracción y generación de texto
6. 066 — Embeddings semánticos y similitud
7. 067 — Reconocimiento automático del habla
8. 068 — Síntesis de voz y clonación responsable
9. 069 — Modelos visión-lenguaje
10. 070 — Fusión multimodal y representación conjunta
11. 071 — Sensores, series y percepción en el borde
12. 072 — Proyecto: asistente multimodal accesible

061 — Clasificación y representación visual

Parte: 05 — Lenguaje, visión, audio e IA multimodal
Nivel: avanzado · **Horas estimadas:** 6
Laboratorio: `perception` · **Estado:** `EXECUTABLE_CORE`

🎯 Propósito

Comprender **clasificación y representación visual** dentro de la evolución de la inteligencia artificial, implementar un experimento mínimo verificable y distinguir qué parte constituye evidencia frente a una afirmación todavía no comprobada.

📚 Resultados de aprendizaje

Al finalizar podrás:

1. Explicar clasificación y representación visual usando los conceptos `visión`, `features`, `clasificación`, `augmentación`.
2. Ejecutar el laboratorio con una semilla explícita y revisar su contrato JSON.
3. Identificar al menos un supuesto, una limitación y un riesgo de aplicación.
4. Comparar el enfoque con la etapa anterior de la ruta de aprendizaje.
5. Producir una evidencia reproducible y una conclusión que no exceda los datos.

🧩 Conceptos centrales

`visión`, `features`, `clasificación`, `augmentación`

🗺️ Ubicación en el mapa de la IA

La visión por computador fue el dominio donde el aprendizaje profundo demostró su superioridad empírica: AlexNet (2012) redujo el error top-5 de ImageNet del 26 % al 16 % y desencadenó la era moderna del deep learning que estudiaste en la parte 04. Esta clase conecta esa historia con la práctica: cómo una imagen se convierte en números, cómo esos números se transforman en **representaciones** útiles y cómo un clasificador decide sobre ellas. Todo lo que sigue en esta parte —detección (062), OCR (063), modelos visión-lenguaje (069)— se construye sobre la idea de representación visual que se formaliza aquí.

📖 Fundamentos

🖼️ La imagen como tensor

Una imagen digital es un tensor de forma `alto × ancho × canales`. Una foto RGB de 224×224 píxeles son 150 528 números enteros en `[0, 255]` (o flotantes normalizados). El problema central de la visión es que ese espacio crudo es **semánticamente inútil**: dos fotos del mismo gato con distinta iluminación distan más,

píxel a píxel, que un gato y una lavadora del mismo color medio. Clasificar exige transformar píxeles en una representación donde la semántica sea geometría: imágenes de la misma clase deben quedar cerca.

🧰 Features diseñadas a mano (era pre-deep)

Antes de 2012 las representaciones se diseñaban manualmente:

- **Bordes y gradientes:** filtros como Sobel responden a cambios bruscos de intensidad.
- **HOG (Histogram of Oriented Gradients):** histograma local de orientaciones de gradiente; base del detector de peatones de Dalal y Triggs (2005).
- **SIFT:** puntos clave invariantes a escala y rotación, con descriptores de 128 dimensiones.

El pipeline clásico era `imagen → detector de features → descriptor → SVM/k-NN`. Funcionaba, pero cada dominio nuevo exigía reingeniería manual de features.

🧠 Features aprendidas: la convolución

Una capa convolucional aprende bancos de filtros pequeños (p. ej. 3×3) que se deslizan sobre la imagen. La operación en cada posición es un producto punto entre el filtro y el parche:

```
salida[i,j] = Σ_u Σ_v  filtro[u,v] · imagen[i+u, j+v]  + sesgo
```

Tres propiedades hacen a la convolución adecuada para imágenes:

1. **Compartición de pesos:** el mismo filtro se aplica en toda la imagen (un detector de bordes sirve en cualquier posición) → muchos menos parámetros que una capa densa.
2. **Localidad:** cada salida depende solo de un vecindario, como los campos receptivos de la corteza visual.
3. **Jerarquía:** apilar capas compone detectores: bordes → texturas → partes → objetos.

La activación de la penúltima capa de una CNN entrenada (p. ej. el vector de 2048 dimensiones de ResNet-50 tras el pooling global) es el **embedding visual**: una representación compacta donde la distancia refleja similitud semántica. Los Vision Transformers (ViT) llegan a representaciones análogas troceando la imagen en parches de 16×16 y aplicando atención.

🎯 Clasificación: de logits a probabilidades

Sobre el embedding `z` se aplica una capa lineal que produce **logits** `s = W·z + b` (un número por clase) y la función softmax los convierte en una distribución:

```
softmax(s)_k = exp(s_k) / Σ_j exp(s_j)
```

La pérdida de entrenamiento es la entropía cruzada `-log p(clase correcta)`. Las métricas de evaluación son accuracy, matriz de confusión y top-k accuracy (la clase correcta está entre las k más probables); con clases desbalanceadas, precisión/recall por clase.

🔁 Augmentación y transferencia

- **Augmentación de datos:** transformar cada imagen de entrenamiento (recortes aleatorios, espejado horizontal, jitter de color) multiplica la diversidad efectiva del dataset y codifica invariancias deseadas ("un gato espejado sigue siendo un gato"). Debe aplicarse solo en entrenamiento, nunca en evaluación, y respetar la semántica (espejar un dígito `6` produce algo que ya no es un `6`).
- **Transfer learning:** una CNN preentrenada en ImageNet ya aprendió features genéricas; para un problema nuevo con pocos datos basta congelar el tronco y reentrenar la cabeza lineal (*linear probing*) o ajustar todo con tasa de aprendizaje baja (*fine-tuning*).

🧮 Ejemplo trabajado

Paso 1 — una convolución a mano. Parche de imagen 3×3 (intensidades) y filtro de borde vertical tipo Sobel:

```
parche:           filtro:
 10  10  90       -1  0  +1
 10  10  90       -2  0  +2
 10  10  90       -1  0  +1

salida = (10·-1)+(10·0)+(90·1)
       + (10·-2)+(10·0)+(90·2)
       + (10·-1)+(10·0)+(90·1)
       = (-10+90) + (-20+180) + (-10+90) = 320
```

El valor alto (320) señala un borde vertical fuerte: a la izquierda oscuro (10), a la derecha claro (90). Sobre una zona uniforme (todo 10) la salida sería 0.

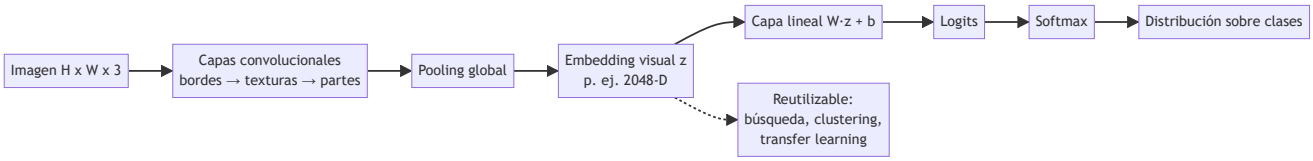
Paso 2 — de logits a decisión. Supón que la cabeza lineal produce logits `s = [2.0, 1.0, 0.1]` para las clases `[gato, perro, coche]`:

```
exp(s) = [7.389, 2.718, 1.105]      suma = 11.212
softmax = [0.659, 0.242, 0.099]
```

Predicción: `gato` con probabilidad 0.659. Nota que softmax **siempre** produce una distribución, incluso ante una imagen de una tostadora: la confianza no es evidencia de que la entrada pertenezca a alguna de las clases conocidas.

📊 Propiedades y comparación

Representación	Origen	Dimensión típica	Datos necesarios	Fortaleza	Límite
HOG / SIFT	Diseño manual	100–3 000	Ninguno (sin entrenamiento)	Interpretable, barata	Tope de precisión bajo
CNN entrenada desde cero	Aprendida	512–2 048	10^5–10^6 imágenes	Estado del arte clásico	Cara de entrenar
CNN preentrenada + linear probe	Transferida	512–2 048	10^2–10^4 imágenes	Muy eficiente en datos	Hereda sesgos del preentrenamiento
ViT preentrenado	Transferida (atención)	768–1 024	10^2–10^4 (fine-tune)	Escala mejor con datos masivos	Débil con poco preentrenamiento



⚠️ Errores conceptuales frecuentes

1. **"La CNN ve como un humano."** Falso: las CNN son muy sensibles a texturas y a perturbaciones adversariales imperceptibles; los humanos priorizan formas. Son sistemas estadísticos, no réplicas de la percepción biológica.
2. **"Softmax alto = el modelo está seguro y tiene razón."** Softmax normaliza logits, no calibra incertidumbre: un modelo puede dar 0.99 a una clase equivocada o a una entrada fuera de distribución que no pertenece a ninguna clase conocida.
3. **"Más augmentación siempre ayuda."** Una augmentación que rompe la semántica de la clase (espejar caracteres, rotar 180° señales de tráfico) enseña invariancias falsas y degrada el desempeño.
4. **"El embedding es objetivo."** El embedding hereda la distribución del preentrenamiento: si ImageNet subrepresenta ciertos contextos culturales, las distancias en el embedding también lo harán.
5. **"Accuracy global basta."** Con 95 % de una clase mayoritaria, un clasificador que siempre predice esa clase logra 95 % de accuracy sin haber aprendido nada útil: hay que mirar la matriz de confusión por clase.

🚀 Del aprendizaje a la operación

Entre este núcleo educativo y un clasificador visual en producción faltan: un pipeline de datos con control de deriva (las cámaras y condiciones cambian con el tiempo), calibración de confianza y umbral de rechazo para entradas fuera de distribución, evaluación por subgrupos para detectar sesgos de desempeño, optimización de inferencia (cuantización, batching) y un circuito de revisión humana para los casos de baja confianza o alto costo de error.

🧪 Laboratorio

```
python lab.py
```

El laboratorio llama a `ai_evolution.labs.run_lab("perception")`. Esta decisión evita una implementación divergente por clase: cada una conserva un entrypoint propio, pero los motores didácticos se prueban como una biblioteca común.

🔍 Evidencia esperada

- tipo de laboratorio y semilla;
- entradas o decisiones observables;
- resultado estructurado;
- lista `evidence` con hechos que pueden inspeccionarse;
- lista `limitations` que impide presentar la demo como producción.

📓 Notebooks

- 📓 `notebook.ipynb`: recorrido guiado con la materia resumida.
- ✍️ `notebook_student.ipynb`: ejercicios para resolver.
- ✅ `notebook_solution.ipynb`: solución de referencia explicada.

📝 Evaluación

Criterio	Peso
Comprensión conceptual	25 %
Ejecución reproducible	25 %
Interpretación basada en evidencia	25 %
Riesgos, límites y mejora propuesta	25 %

Consulta assessment.md para preguntas y criterio de aceptación.

⚠️ Errores comunes

Síntoma	Causa probable	Corrección
El código corre, pero no hay conclusión	Se confundió ejecución con aprendizaje	Explica qué demuestra y qué no demuestra
El resultado cambia sin explicación	No se registró semilla o configuración	Conserva semilla, versión y parámetros
Se promete uso real	Se extrapoló desde una demo educativa	Declara entorno, datos, límites y revisión humana
Se copia una métrica aislada	No existe baseline ni costo de error	Añade comparación y criterio de decisión

❓ Preguntas frecuentes

¿Debo usar una API comercial?
No. El núcleo funciona localmente. Las extensiones LIVE se documentan por separado.

¿El laboratorio representa una implementación industrial?
No por sí solo. Enseña el contrato y el patrón; producción exige integración, seguridad, observabilidad, pruebas y operación.

¿Dónde profundizo?
Revisa las especializaciones enlazadas en el README raíz y la ruta siguiente.

🔗 Referencias

- Szeliski, R. *Computer Vision: Algorithms and Applications* (2e), caps. de reconocimiento — szeliski.org/Book — uso: desarrollo extendido del tema
- Goodfellow, I., Bengio, Y. y Courville, A. *Deep Learning*, cap. 9 (redes convolucionales) — deeplearningbook.org — uso: desarrollo extendido del tema
- Krizhevsky, A., Sutskever, I. y Hinton, G. (2012). "ImageNet Classification with Deep Convolutional Neural Networks" (AlexNet) — NeurIPS 2012 — uso: referencia consultada en su fuente original
- He, K. et al. (2015). "Deep Residual Learning for Image Recognition" (ResNet) — arXiv:1512.03385 — uso: fuente primaria del mecanismo estudiado
- Dosovitskiy, A. et al. (2020). "An Image is Worth 16x16 Words" (ViT) — arXiv:2010.11929 — uso: fuente primaria del mecanismo estudiado
- Documentación oficial de torchvision (modelos y transformaciones) — pytorch.org/vision — uso: referencia consultada en su fuente original

---

📜 Papers que fundamentan esta clase

> Bloque generado por `python scripts/link_papers_to_classes.py`. La fuente es `papers/catalog/papers.json`.

Paper	Año	Qué desbloqueó	Miniatura
P04 · Clasificación de ImageNet con redes neuronales convolucionales profundas	2012	El resultado que convirtió el deep learning en la corriente principal: margen amplio sobre los métodos de visión hechos a mano.	notebook
P44 · Aprendizaje residual profundo para reconocimiento de imágenes	2015	El atajo identidad hace apilables cientos de capas. Es la misma idea aditiva de la LSTM, aplicada a la profundidad.	notebook
P46 · Una imagen vale 16x16 palabras: Transformers para reconocimiento de imágenes a escala	2020	Trata la imagen como una secuencia de parches y aplica un Transformer puro: la convolución deja de ser imprescindible en visión.	notebook

Cada ficha explica el problema anterior, la matemática mínima, los límites y los errores de atribución más frecuentes. Para leerlas con método: cómo leer un paper de IA · anexos matemáticos.

📚 Bibliografía de apoyo

> Bloque generado por `python scripts/link_sources_to_classes.py`. Cada obra lleva su localizador verificado en `sources/bibliography.json`.

Los papers dicen **de dónde salió** el mecanismo. Estas obras lo **desarrollan** con el espacio que una clase no tiene: teoría completa, demostraciones y ejercicios.

Obra	Edición	Localizador	Papel en esta clase
Goodfellow, Ian, Bengio, Yoshua y Courville, Aaron — *Deep Learning*	2016	ISBN 9780262035613 · web de la obra	citada en las referencias de esta clase · cap. 9
Szeliski, Richard — *Computer Vision: Algorithms and Applications*	2.ª · 2022	web de la obra · *pendiente de confirmar en su catálogo*	citada en las referencias de esta clase
Jurafsky, Daniel y Martin, James H. — *Speech and Language Processing*	2.ª (la 3.ª circula como borrador abierto sin ISBN) · 2009	ISBN 9780131873216 · web de la obra	obra de referencia de la parte 05 · lenguaje y habla

📝 Evaluación completa

❓ Preguntas

1. Define clasificación y representación visual sin usar una marca o framework como definición.
2. Explica la relación entre visión, features, clasificación, augmentación.
3. Ejecuta `lab.py` dos veces con la misma semilla. ¿Qué debe conservarse?
4. Identifica una afirmación permitida y una afirmación exagerada sobre el resultado.
5. Propón una prueba negativa o un caso límite.

🏆 Reto verificable

Amplía el resultado del laboratorio con una clave `student_extension` que incluya:

- el supuesto que estás probando;
- una medición o comprobación;
- la conclusión;
- una limitación.

✅ Criterio de aceptación

- [ ] `lab.py` termina con código 0.
- [ ] El resultado contiene `kind`, `seed`, `evidence` y `limitations`.
- [ ] La extensión no modifica el comportamiento de otras clases.
- [ ] La interpretación referencia datos impresos por el laboratorio.
- [ ] Se declara al menos un riesgo o condición de no uso.

062 — Detección, segmentación y pose

Parte: 05 — Lenguaje, visión, audio e IA multimodal
Nivel: avanzado · **Horas estimadas:** 6
Laboratorio: `perception` · **Estado:** `EXECUTABLE_CORE`

🎯 Propósito

Comprender **detección, segmentación y pose** dentro de la evolución de la inteligencia artificial, implementar un experimento mínimo verificable y distinguir qué parte constituye evidencia frente a una afirmación todavía no comprobada.

📚 Resultados de aprendizaje

Al finalizar podrás:

1. Explicar detección, segmentación y pose usando los conceptos `detección`, `segmentación`, `pose`, `tracking`.
2. Ejecutar el laboratorio con una semilla explícita y revisar su contrato JSON.
3. Identificar al menos un supuesto, una limitación y un riesgo de aplicación.
4. Comparar el enfoque con la etapa anterior de la ruta de aprendizaje.
5. Producir una evidencia reproducible y una conclusión que no exceda los datos.

🧩 Conceptos centrales

`detección`, `segmentación`, `pose`, `tracking`

🗺️ Ubicación en el mapa de la IA

Clasificar (clase 061) responde "¿qué hay en la imagen?"; esta clase responde "¿qué hay, **dónde** está y **qué forma** tiene?". La detección de objetos fue el segundo gran dominio conquistado por las CNN (R-CNN, 2014) y su evolución hacia detectores de una sola pasada (YOLO, 2016) habilitó la percepción en tiempo real que exigen la conducción autónoma, la robótica (parte 11) y el análisis de documentos (clase 063). La estimación de pose extiende la misma maquinaria a la localización de articulaciones humanas.

📖 Fundamentos

📦 Tres tareas, tres salidas

Tarea	Salida por objeto	Pregunta
Detección	Caja `(x1, y1, x2, y2)` + clase + confianza	¿Qué y dónde?
Segmentación semántica	Etiqueta de clase por píxel (sin distinguir instancias)	¿Qué clase es cada píxel?
Segmentación de instancias	Máscara binaria por objeto individual	¿Qué píxeles son *este* objeto?
Estimación de pose	Coordenadas de K keypoints (codo, muñeca, …)	¿Dónde están las articulaciones?

En segmentación semántica dos personas contiguas forman una sola mancha "persona"; en segmentación de instancias cada una recibe su propia máscara.

📐 IoU: la métrica de solapamiento

La **Intersection over Union** mide cuánto coincide una caja predicha con la real:

```
IoU(A, B) = área(A ∩ B) / área(A ∪ B)      ∈ [0, 1]
```

Una detección se considera correcta (true positive) si su IoU con una caja real supera un umbral, típicamente 0.5. IoU es invariante a la escala de la imagen y penaliza tanto cajas demasiado pequeñas como demasiado grandes.

🧹 NMS: suprimir duplicados

Los detectores proponen muchas cajas superpuestas para el mismo objeto. La **Non-Maximum Suppression** las depura:

```
1. Ordena las cajas por confianza descendente.
2. Toma la de mayor confianza y muévela a la salida.
3. Elimina toda caja restante con IoU > umbral (p. ej. 0.5) respecto a la elegida.
4. Repite hasta agotar las cajas.
```

Un umbral de NMS demasiado bajo fusiona objetos cercanos distintos; demasiado alto deja duplicados.

🏆 mAP: la métrica de benchmark

Para cada clase se ordenan las detecciones por confianza, se marca cada una como TP o FP (según IoU con las cajas reales, sin reutilizar cajas ya emparejadas), se traza la curva precisión-recall y se calcula el área bajo ella (**AP**). El **mAP** es la media de AP sobre las clases; COCO además promedia sobre umbrales de IoU de 0.5 a 0.95 (`mAP@[.5:.95]`), premiando la localización fina.

🔄 De R-CNN a YOLO: dos filosofías

- **Dos etapas (R-CNN → Fast → Faster R-CNN):** una red propone regiones candidatas y otra las clasifica y refina. Máxima precisión, más latencia. Mask R-CNN añade una rama que predice la máscara de cada región, unificando detección y segmentación de instancias.
- **Una etapa (YOLO, SSD):** la imagen se divide en una grilla y cada celda predice directamente cajas, confianzas y clases en **una sola pasada** de la red. YOLO v1 formuló la detección como regresión: menos precisa en objetos pequeños, pero en tiempo real (45+ fps en 2016).

La estimación de pose moderna predice un **mapa de calor** por keypoint (la posición es el máximo del mapa) y se evalúa con métricas tipo OKS (Object Keypoint Similarity), el análogo de IoU para puntos.

🧮 Ejemplo trabajado

IoU de dos cajas a mano. Caja real `A = (2, 2, 6, 6)` y predicción `B = (4, 4, 8, 8)` (formato `x1, y1, x2, y2`):

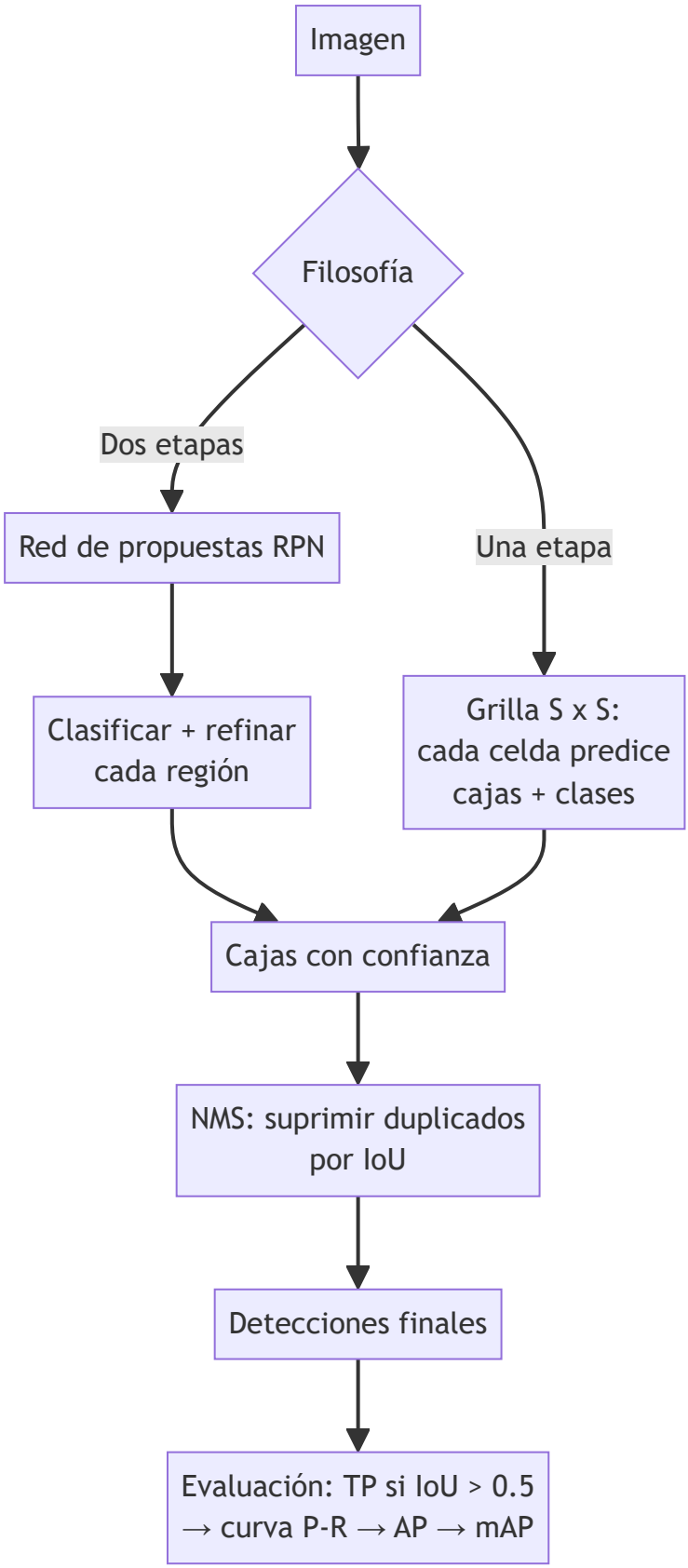
```
Intersección: x ∈ [max(2,4), min(6,8)] = [4, 6] → ancho 2
              y ∈ [max(2,4), min(6,8)] = [4, 6] → alto  2
área(A∩B) = 2 · 2 = 4
área(A) = 4·4 = 16      área(B) = 4·4 = 16
área(A∪B) = 16 + 16 − 4 = 28
IoU = 4 / 28 ≈ 0.143
```

Con umbral 0.5, esta predicción sería un **falso positivo** pese a tocar el objeto.

NMS a mano. Tres detecciones de "perro": `d1` conf 0.9, `d2` conf 0.8 con IoU(d1,d2)=0.7, `d3` conf 0.6 con IoU(d1,d3)=0.1. Con umbral NMS 0.5: se acepta `d1`; `d2` se elimina (0.7 > 0.5, duplicado de d1); `d3` se acepta (0.1 ≤ 0.5, es otro perro). Salida: `{d1, d3}`.

📊 Propiedades y comparación

Detector	Etapas	Velocidad relativa	Precisión relativa	Uso típico
R-CNN (2014)	2 (propuestas externas + CNN por región)	Muy lenta (~47 s/imagen)	Hito histórico	Superada
Faster R-CNN (2015)	2 (RPN aprendida)	Media (~5 fps)	Alta	Precisión primero
Mask R-CNN (2017)	2 + rama de máscara	Media	Alta + máscaras	Segmentación de instancias
YOLO (2016→)	1 (regresión en grilla)	Tiempo real (45+ fps)	Media-alta (mejora por versión)	Latencia primero
DETR (2020)	1 (transformer, sin NMS)	Media	Alta	Elimina heurísticas



⚠️ Errores conceptuales frecuentes

1. **"IoU 0.5 significa que la mitad de la caja está bien."** No: IoU 0.5 exige un solapamiento bastante ajustado — dos cajas iguales desplazadas la mitad de su lado tienen IoU ≈ 0.33, no 0.5.
2. **"Segmentación semántica = segmentación de instancias."** La semántica etiqueta píxeles por clase; dos objetos contiguos de la misma clase se funden. Instancias los separa.
3. **"Más confianza del detector = mejor localización."** La confianza estima la probabilidad de que haya un objeto, no la calidad geométrica de la caja; por eso mAP evalúa ambas cosas por separado (confianza ordena, IoU valida).
4. **"El mAP es comparable entre papers sin mirar el protocolo."** mAP@0.5 (PASCAL) y mAP@[.5:.95] (COCO) difieren en varios puntos; comparar números de protocolos distintos es un error clásico de lectura de benchmarks.
5. **"NMS es inofensivo."** En escenas densas (multitudes, estanterías) NMS elimina detecciones correctas de objetos muy próximos; es una heurística con costo real, y arquitecturas como DETR existen en parte para eliminarla.

🚀 Del aprendizaje a la operación

Un detector operativo exige más que un buen mAP de benchmark: datos anotados del dominio real (las cajas de COCO no cubren tu almacén ni tu quirófano), evaluación por tamaño de objeto y condición de iluminación, calibración del umbral de confianza según el costo de falsos positivos vs falsos negativos, seguimiento (tracking) si hay video, y monitoreo de deriva cuando cambian cámaras, ángulos o estaciones del año.

🧪 Laboratorio

```
python lab.py
```

El laboratorio llama a `ai_evolution.labs.run_lab("perception")`. Esta decisión evita una implementación divergente por clase: cada una conserva un entrypoint propio, pero los motores didácticos se prueban como una biblioteca común.

🔍 Evidencia esperada

- tipo de laboratorio y semilla;
- entradas o decisiones observables;
- resultado estructurado;
- lista `evidence` con hechos que pueden inspeccionarse;
- lista `limitations` que impide presentar la demo como producción.

📓 Notebooks

- 📓 `notebook.ipynb`: recorrido guiado con la materia resumida.
- ✍️ `notebook_student.ipynb`: ejercicios para resolver.
- ✅ `notebook_solution.ipynb`: solución de referencia explicada.

📝 Evaluación

Criterio	Peso
Comprensión conceptual	25 %
Ejecución reproducible	25 %
Interpretación basada en evidencia	25 %
Riesgos, límites y mejora propuesta	25 %

Consulta assessment.md para preguntas y criterio de aceptación.

⚠️ Errores comunes

Síntoma	Causa probable	Corrección
El código corre, pero no hay conclusión	Se confundió ejecución con aprendizaje	Explica qué demuestra y qué no demuestra
El resultado cambia sin explicación	No se registró semilla o configuración	Conserva semilla, versión y parámetros
Se promete uso real	Se extrapoló desde una demo educativa	Declara entorno, datos, límites y revisión humana
Se copia una métrica aislada	No existe baseline ni costo de error	Añade comparación y criterio de decisión

❓ Preguntas frecuentes

¿Debo usar una API comercial?
No. El núcleo funciona localmente. Las extensiones LIVE se documentan por separado.

¿El laboratorio representa una implementación industrial?
No por sí solo. Enseña el contrato y el patrón; producción exige integración, seguridad, observabilidad, pruebas y operación.

¿Dónde profundizo?
Revisa las especializaciones enlazadas en el README raíz y la ruta siguiente.

🔗 Referencias

- Szeliski, R. *Computer Vision: Algorithms and Applications* (2e), cap. de reconocimiento y detección — szeliski.org/Book — uso: desarrollo extendido del tema
- Girshick, R. et al. (2013). "Rich feature hierarchies for accurate object detection" (R-CNN) — arXiv:1311.2524 — uso: fuente primaria del mecanismo estudiado
- Ren, S. et al. (2015). "Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks" — arXiv:1506.01497 — uso: fuente primaria del mecanismo estudiado
- Redmon, J. et al. (2015). "You Only Look Once: Unified, Real-Time Object Detection" — arXiv:1506.02640 — uso: fuente primaria del mecanismo estudiado
- He, K. et al. (2017). "Mask R-CNN" — arXiv:1703.06870 — uso: fuente primaria del mecanismo estudiado

- Dataset y protocolo de evaluación COCO — cocodataset.org — uso: referencia consultada en su fuente original

---

📜 Papers que fundamentan esta clase

> Bloque generado por `python scripts/link_papers_to_classes.py`. La fuente es `papers/catalog/papers.json`.

Paper	Año	Qué desbloqueó	Miniatura
P04 · Clasificación de ImageNet con redes neuronales convolucionales profundas	2012	El resultado que convirtió el deep learning en la corriente principal: margen amplio sobre los métodos de visión hechos a mano.	notebook
P18 · Aprender modelos visuales transferibles con supervisión de lenguaje natural	2021	El texto se convierte en la etiqueta: un solo modelo clasifica categorías que nadie anotó, describiéndolas con palabras.	notebook
P44 · Aprendizaje residual profundo para reconocimiento de imágenes	2015	El atajo identidad hace apilables cientos de capas. Es la misma idea aditiva de la LSTM, aplicada a la profundidad.	notebook

Cada ficha explica el problema anterior, la matemática mínima, los límites y los errores de atribución más frecuentes. Para leerlas con método: cómo leer un paper de IA · anexos matemáticos.

📚 Bibliografía de apoyo

> Bloque generado por `python scripts/link_sources_to_classes.py`. Cada obra lleva su localizador verificado en `sources/bibliography.json`.

Los papers dicen **de dónde salió** el mecanismo. Estas obras lo **desarrollan** con el espacio que una clase no tiene: teoría completa, demostraciones y ejercicios.

Obra	Edición	Localizador	Papel en esta clase
Szeliski, Richard — *Computer Vision: Algorithms and Applications*	2.ª · 2022	web de la obra · *pendiente de confirmar en su catálogo*	citada en las referencias de esta clase
Jurafsky, Daniel y Martin, James H. — *Speech and Language Processing*	2.ª (la 3.ª circula como borrador abierto sin ISBN) · 2009	ISBN 9780131873216 · web de la obra	obra de referencia de la parte 05 · lenguaje y habla

📝 Evaluación completa

❓ Preguntas

1. Define detección, segmentación y pose sin usar una marca o framework como definición.
2. Explica la relación entre detección, segmentación, pose, tracking.
3. Ejecuta `lab.py` dos veces con la misma semilla. ¿Qué debe conservarse?
4. Identifica una afirmación permitida y una afirmación exagerada sobre el resultado.
5. Propón una prueba negativa o un caso límite.

🏆 Reto verificable

Amplía el resultado del laboratorio con una clave `student_extension` que incluya:

- el supuesto que estás probando;
- una medición o comprobación;
- la conclusión;
- una limitación.

✅ Criterio de aceptación

- [ ] `lab.py` termina con código 0.
- [ ] El resultado contiene `kind`, `seed`, `evidence` y `limitations`.
- [ ] La extensión no modifica el comportamiento de otras clases.
- [ ] La interpretación referencia datos impresos por el laboratorio.
- [ ] Se declara al menos un riesgo o condición de no uso.

---

063 — OCR y comprensión de documentos

Parte: 05 — Lenguaje, visión, audio e IA multimodal
Nivel: avanzado · **Horas estimadas:** 6
Laboratorio: `perception` · **Estado:** `EXECUTABLE_CORE`

🎯 Propósito

Comprender **ocr y comprensión de documentos** dentro de la evolución de la inteligencia artificial, implementar un experimento mínimo verificable y distinguir qué parte constituye evidencia frente a una afirmación todavía no comprobada.

📚 Resultados de aprendizaje

Al finalizar podrás:

1. Explicar ocr y comprensión de documentos usando los conceptos `OCR`, `layout`, `tablas`, `documentos`.
2. Ejecutar el laboratorio con una semilla explícita y revisar su contrato JSON.
3. Identificar al menos un supuesto, una limitación y un riesgo de aplicación.
4. Comparar el enfoque con la etapa anterior de la ruta de aprendizaje.
5. Producir una evidencia reproducible y una conclusión que no exceda los datos.

🧩 Conceptos centrales

`OCR`, `layout`, `tablas`, `documentos`

🗺️ Ubicación en el mapa de la IA

El OCR es una de las aplicaciones más antiguas y económicamente relevantes de la visión por computador: convierte imágenes de texto en texto manipulable. Hereda la detección y segmentación de la clase 062 (localizar líneas y palabras es detectar) y anticipa dos ideas clave del programa: la pérdida CTC que reaparece en reconocimiento del habla (clase 067) y los modelos que fusionan texto + layout + imagen (LayoutLM), precursores directos de los modelos visión-lenguaje (clase 069). La comprensión de documentos es hoy la puerta de entrada de la IA a procesos administrativos reales: facturas, formularios, contratos.

📖 Fundamentos

🔤 El pipeline clásico de OCR

```
imagen → preprocesado → análisis de layout → segmentación → reconocimiento → postproceso
```

1. **Preprocesado:** corrección de inclinación (deskew), binarización (Otsu), eliminación de ruido. Un documento escaneado torcido 3° degrada todo lo demás.

2. **Análisis de layout:** separar bloques de texto, tablas, imágenes y encabezados, y determinar el **orden de lectura** (¿dos columnas? ¿tabla?).
3. **Segmentación:** dividir bloques en líneas y, según el motor, en palabras/caracteres.
4. **Reconocimiento:** un modelo secuencial (hoy CNN + RNN/transformer) convierte la imagen de cada línea en una cadena de caracteres.
5. **Postproceso:** diccionarios, modelos de lenguaje y reglas (p. ej. validar formato de fechas o RUT/NIF) corrigen confusiones típicas (`O`↔`0`, `l`↔`1`, `rn`↔`m`).

📝 Reconocimiento sin segmentar caracteres: CTC

Los motores modernos no cortan la línea en caracteres: la red emite una predicción por columna de píxeles y la pérdida **CTC (Connectionist Temporal Classification)** alinea esa secuencia larga con la etiqueta corta. CTC añade un símbolo blanco `∅` y colapsa repeticiones:

```
salida por frames:  c c ∅ a a s ∅ ∅ a
colapso:            c ∅ a s ∅ a  →  "casa"
```

Así la red aprende "qué dice la línea" sin saber dónde empieza cada letra. La misma idea se reutiliza en ASR (clase 067).

📏 Métricas: CER y WER

La calidad se mide con la distancia de edición (Levenshtein) entre la transcripción y la referencia:

```
CER = (S + D + I) / N        S: sustituciones, D: borrados, I: inserciones
                             N: caracteres de la referencia
```

WER es lo mismo a nivel de palabra. Un CER de 1 % suena excelente, pero en un IBAN de 24 caracteres implica ~1 de cada 4 documentos con un dígito erróneo: la métrica debe leerse contra el costo del error por campo.

📄 De OCR a comprensión de documentos

Leer los caracteres no es entender el documento. La **comprensión de documentos** (Document AI) extrae estructura y significado:

- **Extracción clave-valor:** encontrar `total_factura = 1.250,00 €` aunque el importe esté en cualquier posición.
- **Reconocimiento de tablas:** reconstruir filas y columnas con celdas fusionadas.
- **Clasificación de documentos:** ¿factura, contrato, recibo?

Modelos como **LayoutLM** extienden BERT con dos señales extra por token: la **posición 2D** de su caja en la página y (en versiones posteriores) la imagen del recorte. Con ello, la pregunta "¿cuál es el total?" puede resolverse combinando el texto ("Total"), la geometría (el número alineado a su derecha) y el estilo visual (negrita). El preentrenamiento es análogo al de BERT (enmascarar tokens) pero sobre millones de páginas escaneadas.

🧮 Ejemplo trabajado

Referencia: `FACTURA 2024` (12 caracteres, contando el espacio). Salida del OCR: `F4CTURA 224`.

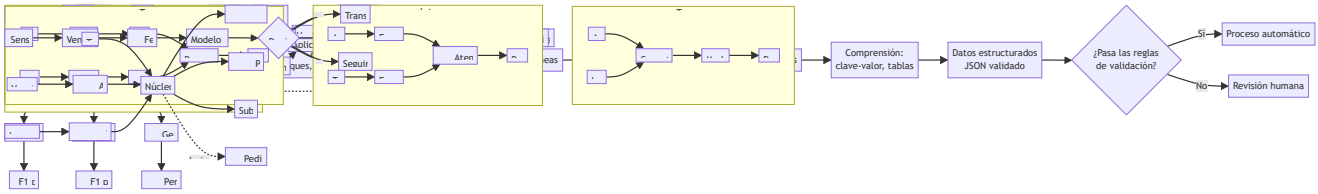
```
Alineación óptima (Levenshtein):
F A C T U R A ␣ 2 0 2 4
F 4 C T U R A ␣ 2 - 2 4
  ^               -
S = 1 (A→4)   D = 1 (falta el 0)   I = 0

CER = (1 + 1 + 0) / 12 ≈ 0.167  → 16.7 %
```

A nivel de palabra: referencia `[FACTURA, 2024]`, hipótesis `[F4CTURA, 224]`: ambas palabras están mal → `WER = 2/2 = 100 %`. Mismo error, dos lecturas: CER moderado, WER catastrófico. Si el campo es el número de factura, el postproceso con la regla `^\d{4}$` detectaría `224` como inválido y enviaría el documento a revisión humana.

📊 Propiedades y comparación

Enfoque	Entrada que usa	Fortaleza	Límite
OCR clásico (Tesseract)	Píxeles binarizados	Local, gratuito, 100+ idiomas	Frágil ante fotos torcidas, manuscritos y layouts complejos
OCR neuronal fin-a-fin (CRNN + CTC)	Imagen de línea	Robusto a fuentes y ruido	Necesita layout externo; sin semántica
LayoutLM y familia	Texto + posición 2D + imagen	Extracción clave-valor y tablas	Requiere fine-tuning anotado por tipo de documento
VLM generalista (clase 069)	Página completa + prompt	Cero configuración inicial	Alucina valores; difícil de auditar campo a campo



⚠️ Errores conceptuales frecuentes

1. **"OCR = comprensión del documento."** OCR produce caracteres; saber cuál de los siete números de la factura es el total es un problema distinto (extracción) que requiere layout y semántica.
2. **"Un CER bajo garantiza datos fiables."** El error no se distribuye uniformemente: se concentra en dígitos, sellos y campos críticos. Hay que medir **exactitud por campo**, no solo CER global.
3. **"El OCR lee en el orden correcto."** El orden de lectura es una decisión del análisis de layout; en documentos a dos columnas o con tablas, un orden equivocado produce texto perfectamente reconocido pero semánticamente revuelto.
4. **"CTC predice dónde está cada carácter."** CTC precisamente evita comprometerse con la posición exacta: alinea secuencias colapsando blancos; las coordenadas por carácter son un subproducto aproximado, no una garantía.

5. **"Con un VLM ya no hace falta OCR."** Los VLM leen texto en imágenes, pero pueden alucinar valores plausibles (un total inventado con formato correcto), algo que un pipeline OCR + validación de reglas no hace silenciosamente.

🚀 Del aprendizaje a la operación

Un sistema de documentos en producción necesita: un conjunto de evaluación por **tipo de documento y por campo** (no un CER global), reglas de validación y umbrales de confianza que deriven a revisión humana los casos dudosos, manejo de versiones de plantillas (los proveedores cambian sus facturas), trazabilidad campo→píxel para auditoría, y cumplimiento de protección de datos cuando los documentos contienen información personal.

🧪 Laboratorio

```
python lab.py
```

El laboratorio llama a `ai_evolution.labs.run_lab("perception")`. Esta decisión evita una implementación divergente por clase: cada una conserva un entrypoint propio, pero los motores didácticos se prueban como una biblioteca común.

🔍 Evidencia esperada

- tipo de laboratorio y semilla;
- entradas o decisiones observables;
- resultado estructurado;
- lista `evidence` con hechos que pueden inspeccionarse;
- lista `limitations` que impide presentar la demo como producción.

📓 Notebooks

- 📓 `notebook.ipynb`: recorrido guiado con la materia resumida.
- ✍️ `notebook_student.ipynb`: ejercicios para resolver.
- ✅ `notebook_solution.ipynb`: solución de referencia explicada.

📝 Evaluación

Criterio	Peso
Comprensión conceptual	25 %
Ejecución reproducible	25 %
Interpretación basada en evidencia	25 %
Riesgos, límites y mejora propuesta	25 %

Consulta assessment.md para preguntas y criterio de aceptación.

⚠️ Errores comunes

Síntoma	Causa probable	Corrección
El código corre, pero no hay conclusión	Se confundió ejecución con aprendizaje	Explica qué demuestra y qué no demuestra
El resultado cambia sin explicación	No se registró semilla o configuración	Conserva semilla, versión y parámetros
Se promete uso real	Se extrapoló desde una demo educativa	Declara entorno, datos, límites y revisión humana
Se copia una métrica aislada	No existe baseline ni costo de error	Añade comparación y criterio de decisión

❓ Preguntas frecuentes

¿Debo usar una API comercial?
No. El núcleo funciona localmente. Las extensiones LIVE se documentan por separado.

¿El laboratorio representa una implementación industrial?
No por sí solo. Enseña el contrato y el patrón; producción exige integración, seguridad, observabilidad, pruebas y operación.

¿Dónde profundizo?
Revisa las especializaciones enlazadas en el README raíz y la ruta siguiente.

🔗 Referencias

- Documentación oficial de Tesseract OCR — tesseract-ocr.github.io — uso: referencia consultada en su fuente original
- Graves, A. et al. (2006). "Connectionist Temporal Classification: Labelling Unsegmented Sequence Data with Recurrent Neural Networks" (ICML 2006) — cs.toronto.edu/~graves/icml_2006.pdf — uso: referencia consultada en su fuente original
- Shi, B. et al. (2015). "An End-to-End Trainable Neural Network for Image-based Sequence Recognition" (CRNN) — arXiv:1507.05717 — uso: fuente primaria del mecanismo estudiado
- Xu, Y. et al. (2019). "LayoutLM: Pre-training of Text and Layout for Document Image Understanding" — arXiv:1912.13318 — uso: fuente primaria del mecanismo estudiado
- Szeliski, R. *Computer Vision: Algorithms and Applications* (2e) — szeliski.org/Book — uso: desarrollo extendido del tema
- Hugging Face Tasks: Document Question Answering — huggingface.co/tasks/document-question-answering — uso: referencia consultada en su fuente original

---

📜 Papers que fundamentan esta clase

> Bloque generado por `python scripts/link_papers_to_classes.py`. La fuente es

> `papers/catalog/papers.json`.

Paper	Año	Qué desbloqueó	Miniatura
P125 · LayoutLM: preentrenamiento de texto y disposición para comprensión de documentos	2020	Añade la posición en la página como una incrustación más, y con eso convierte un modelo de lenguaje en un lector de formularios y facturas.	notebook
P126 · Transformer de comprensión de documentos sin OCR	2022	Va de la imagen del documento a la salida estructurada sin pasar por OCR, y con ello elimina una fuente de error que la etapa siguiente no podía corregir.	notebook

Cada ficha explica el problema anterior, la matemática mínima, los límites y los errores de atribución más frecuentes. Para leerlas con método: cómo leer un paper de IA · anexos matemáticos.

📚 Bibliografía de apoyo

> Bloque generado por `python scripts/link_sources_to_classes.py`. Cada obra lleva su localizador verificado en `sources/bibliography.json`.

Los papers dicen **de dónde salió** el mecanismo. Estas obras lo **desarrollan** con el espacio que una clase no tiene: teoría completa, demostraciones y ejercicios.

Obra	Edición	Localizador	Papel en esta clase
Szeliski, Richard — *Computer Vision: Algorithms and Applications*	2.ª · 2022	web de la obra · *pendiente de confirmar en su catálogo*	citada en las referencias de esta clase
Jurafsky, Daniel y Martin, James H. — *Speech and Language Processing*	2.ª (la 3.ª circula como borrador abierto sin ISBN) · 2009	ISBN 9780131873216 · web de la obra	obra de referencia de la parte 05 · lenguaje y habla

📝 Evaluación completa

❓ Preguntas

1. Define ocr y comprensión de documentos sin usar una marca o framework como definición.
2. Explica la relación entre OCR, layout, tablas, documentos.
3. Ejecuta `lab.py` dos veces con la misma semilla. ¿Qué debe conservarse?
4. Identifica una afirmación permitida y una afirmación exagerada sobre el resultado.
5. Propón una prueba negativa o un caso límite.

🏆 Reto verificable

Amplía el resultado del laboratorio con una clave `student_extension` que incluya:

- el supuesto que estás probando;
- una medición o comprobación;
- la conclusión;
- una limitación.

✅ Criterio de aceptación

- [ ] `lab.py` termina con código 0.
- [ ] El resultado contiene `kind`, `seed`, `evidence` y `limitations`.
- [ ] La extensión no modifica el comportamiento de otras clases.
- [ ] La interpretación referencia datos impresos por el laboratorio.
- [ ] Se declara al menos un riesgo o condición de no uso.

---

064 — Tokenización y representación del lenguaje

Parte: 05 — Lenguaje, visión, audio e IA multimodal
Nivel: avanzado · **Horas estimadas:** 6
Laboratorio: `llm` · **Estado:** `EXECUTABLE_CORE`

🎯 Propósito

Comprender **tokenización y representación del lenguaje** dentro de la evolución de la inteligencia artificial, implementar un experimento mínimo verificable y distinguir qué parte constituye evidencia frente a una afirmación todavía no comprobada.

📚 Resultados de aprendizaje

Al finalizar podrás:

1. Explicar tokenización y representación del lenguaje usando los conceptos `tokens`, `vocabulario`, `subwords`, `embeddings`.
2. Ejecutar el laboratorio con una semilla explícita y revisar su contrato JSON.
3. Identificar al menos un supuesto, una limitación y un riesgo de aplicación.
4. Comparar el enfoque con la etapa anterior de la ruta de aprendizaje.
5. Producir una evidencia reproducible y una conclusión que no exceda los datos.

🧩 Conceptos centrales

`tokens`, `vocabulario`, `subwords`, `embeddings`

🗺️ Ubicación en el mapa de la IA

Todo sistema de PLN —del clasificador de spam al LLM de frontera— empieza por la misma decisión: cómo convertir texto en unidades discretas que un modelo pueda procesar. La tokenización por subpalabras (BPE, WordPiece) resolvió hacia 2016 el dilema entre vocabularios inmanejables y pérdida de palabras desconocidas, y es la puerta de entrada de GPT, BERT y todos los modelos de la parte 06. Esta clase también recorre la evolución de la **representación**: de la bolsa de palabras dispersa a los embeddings densos que se profundizan en la clase 066.

📖 Fundamentos

🎗️ El problema de la tokenización

Un modelo opera sobre un **vocabulario finito** de símbolos. Las opciones extremas fallan:

- **Por palabras:** vocabulario enorme (el español tiene millones de formas flexionadas), y toda palabra no vista se convierte en un token desconocido `<UNK>` que destruye información (`"fotosíntesis" → <UNK>`).

- **Por caracteres:** vocabulario mínimo (~100 símbolos) y sin `<UNK>`, pero secuencias larguísimas y cada símbolo casi sin significado.

Las **subpalabras** son el punto medio: palabras frecuentes quedan enteras (`"casa"` → 1 token) y las raras se descomponen en fragmentos reutilizables (`"fotosíntesis"` → `foto + síntesis`). Nunca hay `<UNK>`: en el peor caso se cae a caracteres.

🔁 BPE: Byte-Pair Encoding

BPE (Sennrich et al., 2016) aprende el vocabulario por fusiones sucesivas:

```
1. Vocabulario inicial: todos los caracteres del corpus.
2. Cuenta la frecuencia de cada PAR de símbolos adyacentes.
3. Fusiona el par más frecuente en un símbolo nuevo y regístralo como regla.
4. Repite hasta alcanzar el tamaño de vocabulario deseado (p. ej. 50 000).
```

Para tokenizar texto nuevo se aplican las reglas de fusión **en el orden aprendido**. GPT-2 y sucesores usan BPE a nivel de **bytes**: el vocabulario base son los 256 bytes, así cualquier texto (emojis, chino, código) es tokenizable sin excepciones.

WordPiece (BERT) es similar pero elige la fusión que maximiza la verosimilitud del corpus, no la más frecuente, y marca los fragmentos no iniciales con `##` (`jugando → ju + ##gando`). **SentencePiece** trata el espacio como un símbolo más (`▁`), eliminando la dependencia de un pre-tokenizador por idioma.

🧮 De tokens a números: representaciones

- **One-hot:** cada token es un vector de tamaño |V| con un único 1. Sin noción de similitud: `perro` y `can` son ortogonales.
- **Bolsa de palabras (BoW):** un documento es la suma de sus one-hots (conteos por término). Ignora el orden: "el perro muerde al niño" ≡ "el niño muerde al perro".
- **TF-IDF:** pondera cada conteo por lo raro que es el término en el corpus, restando peso a palabras omnipresentes ("el", "de") — se calcula en detalle en la clase 065.
- **Embeddings densos:** cada token del vocabulario tiene un vector aprendido de d dimensiones (d ≈ 100–4096). La capa de embedding es una tabla `|V| × d` que se entrena con el resto del modelo; tokens que aparecen en contextos similares acaban cerca (clase 066 profundiza).

🪚 Consecuencias prácticas de la tokenización

- El **costo** de una API de LLM y su **ventana de contexto** se miden en tokens, no en palabras: en español ~1.4–1.8 tokens por palabra en tokenizadores entrenados con sesgo hacia el inglés.
- La aritmética con dígitos sufre porque `12345` puede partirse en `123 + 45`.
- Idiomas subrepresentados en el corpus de entrenamiento del tokenizador se fragmentan más → más tokens → más costo y peor rendimiento: la tokenización codifica una desigualdad silenciosa entre idiomas.

🧮 Ejemplo trabajado

Corpus de juguete (con frecuencias): `low ×5, lower ×2, newest ×6, widest ×3`. Cada palabra termina en el marcador `_`. Aprendamos las primeras fusiones BPE:

```
Símbolos iniciales: l o w _ e r n s t w i d

Conteo de pares (ponderado por frecuencia de palabra):
  (e,s): newest 6 + widest 3 = 9   ← máximo
  (s,t): 9    (t,_): 9   (l,o): 7   (o,w): 7 ...

Fusión 1: e+s → es      newest = n e w es t _
Fusión 2: es+t → est    newest = n e w est _   widest = w i d est _
Fusión 3: est+_ → est_  (frecuencia 9)
Fusión 4: l+o → lo      (5+2 = 7)
Fusión 5: lo+w → low    low_ = low _
```

Con solo 5 fusiones, el sufijo superlativo `est_` ya es un token propio: BPE descubrió morfología sin reglas lingüísticas. Una palabra nueva como `lowest` se tokenizaría `low + est_` — dos unidades con significado, ninguna `<UNK>`.

📊 Propiedades y comparación

Estrategia	Tamaño de vocabulario	`<UNK>`	Longitud de secuencia	Usada por
Palabras	10^5–10^7	Sí, frecuente	Corta	PLN clásico
Caracteres	~10^2	No	Muy larga	Modelos char-level
BPE (bytes)	$3{\cdot}10^4$–10^5	Nunca	Media	GPT-2/3/4, Llama
WordPiece	~$3{\cdot}10^4$	Raro (`[UNK]` existe)	Media	BERT
SentencePiece/Unigram	$3{\cdot}10^4$–$2.5{\cdot}10^5$	No	Media	T5, Gemma, multilingües

⚠️ Errores conceptuales frecuentes

1. **"Un token = una palabra."** Solo las palabras frecuentes son un token; `fotosíntesis` pueden ser 3–5 tokens y un emoji hasta 3. Presupuestar contexto o costo contando palabras subestima sistemáticamente.
2. **"El tokenizador entiende morfología."** BPE solo optimiza frecuencia: a menudo corta `est` como sufijo, pero también produce cortes sin sentido lingüístico (`desagradable → desa + grad + able`). La

coincidencia con la morfología es estadística, no diseñada.

3. **"El tokenizador es intercambiable."** Los IDs solo tienen sentido para el modelo entrenado con ese tokenizador exacto: pasar IDs de BERT a GPT produce basura. Modelo y tokenizador forman una pareja indivisible.
4. **"BoW y embeddings son intercambiables."** BoW pierde el orden y no captura sinonimia; los embeddings capturan similitud pero pierden interpretabilidad por término. La elección depende de la tarea (clase 065).
5. **"Los embeddings de la tabla inicial ya son semánticos."** Al inicio del entrenamiento son aleatorios; la semántica emerge del objetivo de entrenamiento. Un embedding sin entrenar no acerca `perro` a `can`.

🚀 Del aprendizaje a la operación

En producción la tokenización aparece en los presupuestos: estimar costos de API exige contar tokens con el tokenizador real del modelo (no palabras), la ventana de contexto se gestiona en tokens, y los sistemas multilingües deben medir la "tasa de fertilidad" (tokens/palabra) por idioma para detectar usuarios penalizados. Además, versionar el tokenizador es crítico: cambiar una regla de fusión invalida embeddings cacheados e índices construidos con la versión anterior.

🧪 Laboratorio

```
python lab.py
```

El laboratorio llama a `ai_evolution.labs.run_lab("llm")`. Esta decisión evita una implementación divergente por clase: cada una conserva un entrypoint propio, pero los motores didácticos se prueban como una biblioteca común.

🔍 Evidencia esperada

- tipo de laboratorio y semilla;
- entradas o decisiones observables;
- resultado estructurado;
- lista `evidence` con hechos que pueden inspeccionarse;
- lista `limitations` que impide presentar la demo como producción.

📓 Notebooks

- 📓 `notebook.ipynb`: recorrido guiado con la materia resumida.
- ✍️ `notebook_student.ipynb`: ejercicios para resolver.
- ✅ `notebook_solution.ipynb`: solución de referencia explicada.

📝 Evaluación

Criterio	Peso
Comprensión conceptual	25 %
Ejecución reproducible	25 %
Interpretación basada en evidencia	25 %
Riesgos, límites y mejora propuesta	25 %

Consulta assessment.md para preguntas y criterio de aceptación.

⚠️ Errores comunes

Síntoma	Causa probable	Corrección
El código corre, pero no hay conclusión	Se confundió ejecución con aprendizaje	Explica qué demuestra y qué no demuestra
El resultado cambia sin explicación	No se registró semilla o configuración	Conserva semilla, versión y parámetros
Se promete uso real	Se extrapoló desde una demo educativa	Declara entorno, datos, límites y revisión humana
Se copia una métrica aislada	No existe baseline ni costo de error	Añade comparación y criterio de decisión

❓ Preguntas frecuentes

¿Debo usar una API comercial?
No. El núcleo funciona localmente. Las extensiones LIVE se documentan por separado.

¿El laboratorio representa una implementación industrial?
No por sí solo. Enseña el contrato y el patrón; producción exige integración, seguridad, observabilidad, pruebas y operación.

¿Dónde profundizo?
Revisa las especializaciones enlazadas en el README raíz y la ruta siguiente.

🔗 Referencias

- Jurafsky, D. y Martin, J. H. *Speech and Language Processing* (3e), cap. 2 (texto, tokenización y BPE) — web.stanford.edu/~jurafsky/slp3 — uso: desarrollo extendido del tema
- Sennrich, R., Haddow, B. y Birch, A. (2015). "Neural Machine Translation of Rare Words with Subword Units" (BPE) — arXiv:1508.07909 — uso: fuente primaria del mecanismo estudiado
- Wu, Y. et al. (2016). "Google's Neural Machine Translation System" (WordPiece en producción) — arXiv:1609.08144 — uso: fuente primaria del mecanismo estudiado
- Kudo, T. y Richardson, J. (2018). "SentencePiece: A simple and language independent subword tokenizer" — arXiv:1808.06226 — uso: fuente primaria del mecanismo estudiado
- Documentación oficial de Hugging Face Tokenizers — huggingface.co/docs/tokenizers — uso: referencia consultada en su fuente original

📜 Papers que fundamentan esta clase

> Bloque generado por `python scripts/link_papers_to_classes.py`. La fuente es `papers/catalog/papers.json`.

Paper	Año	Qué desbloqueó	Miniatura
P05 · Estimación eficiente de representaciones de palabras en un espacio vectorial	2013	El significado distribucional se vuelve barato: vectores densos entrenables sobre miles de millones de palabras.	notebook
P23 · GloVe: vectores globales para representación de palabras	2014	Unifica las dos familias de embeddings: factorizar estadísticas globales de co-ocurrencia con la ventaja de los métodos predictivos.	notebook

Cada ficha explica el problema anterior, la matemática mínima, los límites y los errores de atribución más frecuentes. Para leerlas con método: cómo leer un paper de IA · anexos matemáticos.

📚 Bibliografía de apoyo

> Bloque generado por `python scripts/link_sources_to_classes.py`. Cada obra lleva su localizador verificado en `sources/bibliography.json`.

Los papers dicen **de dónde salió** el mecanismo. Estas obras lo **desarrollan** con el espacio que una clase no tiene: teoría completa, demostraciones y ejercicios.

Obra	Edición	Localizador	Papel en esta clase
Jurafsky, Daniel y Martin, James H. — *Speech and Language Processing*	2.ª (la 3.ª circula como borrador abierto sin ISBN) · 2009	ISBN 9780131873216 · web de la obra	citada en las referencias de esta clase · cap. 2 · obra de referencia de la parte 05

📝 Evaluación completa

❓ Preguntas

1. Define tokenización y representación del lenguaje sin usar una marca o framework como definición.
2. Explica la relación entre tokens, vocabulario, subwords, embeddings.
3. Ejecuta `lab.py` dos veces con la misma semilla. ¿Qué debe conservarse?
4. Identifica una afirmación permitida y una afirmación exagerada sobre el resultado.
5. Propón una prueba negativa o un caso límite.

🏆 Reto verificable

Amplía el resultado del laboratorio con una clave `student_extension` que incluya:

- el supuesto que estás probando;
- una medición o comprobación;
- la conclusión;
- una limitación.

✅ Criterio de aceptación

- [ ] `lab.py` termina con código 0.
- [ ] El resultado contiene `kind`, `seed`, `evidence` y `limitations`.
- [ ] La extensión no modifica el comportamiento de otras clases.
- [ ] La interpretación referencia datos impresos por el laboratorio.
- [ ] Se declara al menos un riesgo o condición de no uso.

---

065 — Clasificación, extracción y generación de texto

Parte: 05 — Lenguaje, visión, audio e IA multimodal
Nivel: avanzado · **Horas estimadas:** 6
Laboratorio: `llm` · **Estado:** `EXECUTABLE_CORE`

🎯 Propósito

Comprender **clasificación, extracción y generación de texto** dentro de la evolución de la inteligencia artificial, implementar un experimento mínimo verificable y distinguir qué parte constituye evidencia frente a una afirmación todavía no comprobada.

📚 Resultados de aprendizaje

Al finalizar podrás:

1. Explicar clasificación, extracción y generación de texto usando los conceptos `NLP`, `NER`, `clasificación`, `generación`.
2. Ejecutar el laboratorio con una semilla explícita y revisar su contrato JSON.
3. Identificar al menos un supuesto, una limitación y un riesgo de aplicación.
4. Comparar el enfoque con la etapa anterior de la ruta de aprendizaje.
5. Producir una evidencia reproducible y una conclusión que no exceda los datos.

🧩 Conceptos centrales

`NLP`, `NER`, `clasificación`, `generación`

🗺️ Ubicación en el mapa de la IA

Con el texto ya tokenizado (clase 064), esta clase recorre las tres familias de tareas que estructuran el PLN aplicado: **clasificar** (asignar una etiqueta al texto), **extraer** (localizar entidades y relaciones dentro del texto) y **generar** (producir texto nuevo). Son las tareas que el PLN estadístico resolvía con TF-IDF, HMM y n-gramas, y que los LLM de la parte 06 resuelven hoy con un solo modelo. Entender las versiones clásicas —y sus métricas: F1 por entidad, perplejidad— es lo que permite evaluar con rigor a los modelos modernos.

📖 Fundamentos

🏷️ Clasificación de texto

Asignar una etiqueta a un documento: spam/no-spam, sentimiento, tema, idioma. El pipeline clásico sigue siendo un baseline serio:

```
texto → tokens → vector TF-IDF → clasificador lineal (regresión logística / SVM)
```

TF-IDF pondera cada término t en el documento d:

```
tf(t, d)  = conteo de t en d (o su versión logarítmica)
idf(t)    = log(N / df(t))     N: nº de documentos, df: nº que contienen t
tfidf     = tf · idf
```

Un término frecuente en el documento pero raro en el corpus (alto tf, alto idf) es distintivo; "el" aparece en todos los documentos → idf ≈ 0 → peso nulo. La evaluación usa precisión, recall y F1 **por clase** (con clases desbalanceadas, la accuracy engaña) y la media macro (todas las clases pesan igual) o micro (todos los ejemplos pesan igual).

🔍 Extracción: NER y etiquetado de secuencias

El **reconocimiento de entidades nombradas** (NER) localiza spans con tipo: persona, organización, lugar, fecha, importe. Se modela como etiquetado token a token con el esquema **BIO**: `B-TIPO` abre una entidad, `I-TIPO` la continúa, `O` es "fuera":

```
Gabriela  Mistral  ganó   el  Nobel   en  1945
B-PER     I-PER    O      O   B-MISC  O   B-DATE
```

Los modelos clásicos (HMM, CRF) explotan que las etiquetas vecinas se restringen entre sí (`I-PER` no puede seguir a `O`); los modernos ponen un clasificador por token sobre un encoder tipo BERT. La métrica es **F1 a nivel de entidad**: una entidad cuenta como acierto solo si el span completo y el tipo coinciden — `Gabriela` sola, con `Mistral` fuera, es un error, no medio acierto.

✍️ Generación: modelos de lenguaje y perplejidad

Un **modelo de lenguaje** asigna probabilidad a secuencias, factorizada token a token:

```
P(w1 … wn) = Π_i P(wi | w1 … w(i-1))
```

Los n-gramas truncan el contexto a n−1 palabras; los transformers lo extienden a miles de tokens. Generar = muestrear iterativamente de `P(siguiente | contexto)` (con temperatura, top-k o top-p controlando la aleatoriedad). La métrica intrínseca es la **perplejidad**:

```
PP = P(w1 … wn) ^ (-1/n)
```

Es el "factor de ramificación efectivo": PP = 20 significa que, en promedio, el modelo duda entre ~20 opciones equiprobables por token. Menor es mejor, pero solo es comparable entre modelos con el **mismo tokenizador y el mismo corpus de prueba**. La calidad de la generación para humanos exige además métricas extrínsecas (evaluación humana, tasas de error factual), porque una perplejidad baja no garantiza texto veraz ni útil.

⚖️ Clásico vs LLM

Para clasificación con miles de ejemplos etiquetados, TF-IDF + regresión logística sigue siendo rápido, barato, interpretable (pesos por término) y difícil de batir por poco margen. El LLM gana cuando hay pocos o cero

ejemplos, cuando la tarea exige comprensión profunda, o cuando el espacio de salida es abierto (generación, resumen). La decisión es de ingeniería: costo por inferencia, latencia, auditabilidad y deriva.

🧮 Ejemplo trabajado

TF-IDF de un corpus de 3 documentos (tf = conteo crudo, idf = log natural):

```
d1: "el gato duerme"
d2: "el perro ladra"
d3: "el gato juega con el perro"

df: el=3, gato=2, perro=2, duerme=1, ladra=1, juega=1, con=1
idf: el    = ln(3/3) = 0
     gato  = ln(3/2) ≈ 0.405
     perro = ln(3/2) ≈ 0.405
     duerme = ladra = juega = con = ln(3/1) ≈ 1.099

TF-IDF de d3 (conteos: el=2, gato=1, juega=1, con=1, perro=1):
  el    → 2 · 0     = 0
  gato  → 1 · 0.405 = 0.405
  perro → 1 · 0.405 = 0.405
  juega → 1 · 1.099 = 1.099
  con   → 1 · 1.099 = 1.099
```

`el` desaparece pese a aparecer dos veces; `juega` y `con` dominan por raros. Un clasificador lineal sobre estos vectores aprendería que `duerme` / `juega` discriminan actividades y que `el` no aporta nada.

Perplejidad mínima. Si un modelo asigna a la frase de 4 tokens probabilidades `0.5 · 0.25 · 0.5 · 0.125 = 0.00390625`, entonces `PP = 0.00390625^(-1/4) = (2^-8)^(-1/4) = 2^2 = 4`: el modelo duda en promedio entre 4 opciones por token.

📊 Propiedades y comparación

Tarea	Enfoque clásico	Enfoque neuronal/LLM	Métrica	Cuándo basta lo clásico
Clasificación	TF-IDF + lineal	Fine-tune BERT / prompt LLM	F1 macro	Miles de etiquetas, dominio estable
NER	CRF con features	Encoder + clasificador por token	F1 por entidad	Tipos estándar, texto formal
Generación	n-gramas	Transformer autoregresivo	Perplejidad + eval. humana	Prácticamente nunca (n-gramas solo como baseline)

⚠️ Errores conceptuales frecuentes

1. **"Accuracy alta = buen clasificador."** Con 95 % de correos legítimos, predecir siempre "no spam" da 95 % de accuracy y 0 % de recall de spam. Con desbalance, mirar F1 por clase.
2. **"En NER, acertar la mayoría de los tokens es acertar."** La métrica estándar es por entidad completa: etiquetar `B-PER` en `Gabriela` pero `O` en `Mistral` cuenta como fallo total del span. El F1 por token infla el resultado.
3. **"Menor perplejidad = mejor texto."** La perplejidad mide ajuste distribucional al corpus de prueba, no veracidad, coherencia global ni utilidad; y no es comparable entre tokenizadores distintos.
4. **"La generación del modelo es una consulta a una base de datos."** Muestrear de `P(siguiente | contexto)` produce texto plausible, no hechos verificados: la fluidez es exactamente lo que hace peligrosa la alucinación.
5. **"El LLM siempre supera al baseline."** Sin medir, no: en clasificación con datos abundantes y dominio cerrado, un TF-IDF + lineal bien ajustado empata o gana con una fracción del costo y con pesos auditables.

🚀 Del aprendizaje a la operación

Operar estas tareas exige: un conjunto de prueba congelado y versionado por tarea, F1 por clase/entidad con umbrales de alarma, monitoreo de deriva de vocabulario (el spam de hoy no usa las palabras de 2020), un baseline clásico permanente como control de costo-beneficio frente al LLM, y para generación, evaluación humana muestreada y detección de contenido inventado antes de exponer el texto a usuarios.

🧪 Laboratorio

```
python lab.py
```

El laboratorio llama a `ai_evolution.labs.run_lab("llm")`. Esta decisión evita una implementación divergente por clase: cada una conserva un entrypoint propio, pero los motores didácticos se prueban como una biblioteca común.

🔍 Evidencia esperada

- tipo de laboratorio y semilla;
- entradas o decisiones observables;
- resultado estructurado;

- lista `evidence` con hechos que pueden inspeccionarse;
- lista `limitations` que impide presentar la demo como producción.

📓 Notebooks

- 📓 `notebook.ipynb`: recorrido guiado con la materia resumida.
- ✍️ `notebook_student.ipynb`: ejercicios para resolver.
- ✅ `notebook_solution.ipynb`: solución de referencia explicada.

📝 Evaluación

Criterio	Peso
Comprensión conceptual	25 %
Ejecución reproducible	25 %
Interpretación basada en evidencia	25 %
Riesgos, límites y mejora propuesta	25 %

Consulta assessment.md para preguntas y criterio de aceptación.

⚠️ Errores comunes

Síntoma	Causa probable	Corrección
El código corre, pero no hay conclusión	Se confundió ejecución con aprendizaje	Explica qué demuestra y qué no demuestra
El resultado cambia sin explicación	No se registró semilla o configuración	Conserva semilla, versión y parámetros
Se promete uso real	Se extrapoló desde una demo educativa	Declara entorno, datos, límites y revisión humana
Se copia una métrica aislada	No existe baseline ni costo de error	Añade comparación y criterio de decisión

❓ Preguntas frecuentes

¿Debo usar una API comercial?
No. El núcleo funciona localmente. Las extensiones LIVE se documentan por separado.

¿El laboratorio representa una implementación industrial?
No por sí solo. Enseña el contrato y el patrón; producción exige integración, seguridad, observabilidad, pruebas y operación.

¿Dónde profundizo?
Revisa las especializaciones enlazadas en el README raíz y la ruta siguiente.

🔗 Referencias

- Jurafsky, D. y Martin, J. H. *Speech and Language Processing* (3e): caps. de n-gramas, clasificación con Naive Bayes/regresión logística y etiquetado de secuencias — web.stanford.edu/~jurafsky/slp3 — uso: desarrollo extendido del tema
- Manning, C., Raghavan, P. y Schütze, H. *Introduction to Information Retrieval*, cap. 6 (TF-IDF y modelo vectorial) — nlp.stanford.edu/IR-book — uso: referencia consultada en su fuente original
- Devlin, J. et al. (2018). "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding" — arXiv:1810.04805 — uso: fuente primaria del mecanismo estudiado
- Documentación de scikit-learn: Working with text data — scikit-learn.org — uso: referencia consultada en su fuente original
- Documentación de spaCy: Linguistic features (NER) — spacy.io/usage/linguistic-features — uso: referencia consultada en su fuente original

---

📜 Papers que fundamentan esta clase

> Bloque generado por `python scripts/link_papers_to_classes.py`. La fuente es `papers/catalog/papers.json`.

Paper	Año	Qué desbloqueó	Miniatura
P09 · BERT: preentrenamiento de Transformers bidireccionales profundos para comprensión del lenguaje	2018	Consolida el patrón preentrenar-y-ajustar: un mismo modelo base sirve para muchas tareas con un ajuste pequeño.	notebook
P24 · Representaciones profundas de palabras dependientes del contexto	2018	Un vector por APARICIÓN y no por palabra: la polisemia deja de colapsar en un único punto del espacio.	notebook
P25 · Explorar los límites del aprendizaje por transferencia con un Transformer unificado texto a texto	2019	Todo problema de texto se reescribe como texto → texto: un solo modelo, una sola pérdida, cero cabezas específicas.	notebook

Cada ficha explica el problema anterior, la matemática mínima, los límites y los errores de atribución más frecuentes. Para leerlas con método: cómo leer un paper de IA · anexos matemáticos.

📚 Bibliografía de apoyo

> Bloque generado por `python scripts/link_sources_to_classes.py`. Cada obra lleva su localizador verificado en `sources/bibliography.json`.

Los papers dicen **de dónde salió** el mecanismo. Estas obras lo **desarrollan** con el espacio que una clase no tiene: teoría completa, demostraciones y ejercicios.

Obra	Edición	Localizador	Papel en esta clase
Jurafsky, Daniel y Martin, James H. — *Speech and Language Processing*	2.ª (la 3.ª circula como borrador abierto sin ISBN) · 2009	ISBN 9780131873216 · web de la obra	citada en las referencias de esta clase · obra de referencia de la parte 05
Manning, Christopher D., Raghavan, Prabhakar y Schütze, Hinrich — *Introduction to Information Retrieval*	2008	ISBN 9780521865715 · web de la obra	citada en las referencias de esta clase · cap. 6

📝 Evaluación completa

❓ Preguntas

1. Define clasificación, extracción y generación de texto sin usar una marca o framework como definición.
2. Explica la relación entre NLP, NER, clasificación, generación.
3. Ejecuta `lab.py` dos veces con la misma semilla. ¿Qué debe conservarse?
4. Identifica una afirmación permitida y una afirmación exagerada sobre el resultado.
5. Propón una prueba negativa o un caso límite.

🏆 Reto verificable

Amplía el resultado del laboratorio con una clave `student_extension` que incluya:

- el supuesto que estás probando;
- una medición o comprobación;
- la conclusión;
- una limitación.

✅ Criterio de aceptación

- [ ] `lab.py` termina con código 0.
- [ ] El resultado contiene `kind`, `seed`, `evidence` y `limitations`.
- [ ] La extensión no modifica el comportamiento de otras clases.
- [ ] La interpretación referencia datos impresos por el laboratorio.
- [ ] Se declara al menos un riesgo o condición de no uso.

---

066 — Embeddings semánticos y similitud

Parte: 05 — Lenguaje, visión, audio e IA multimodal
Nivel: avanzado · **Horas estimadas:** 6
Laboratorio: `retrieval` · **Estado:** `EXECUTABLE_CORE`

🎯 Propósito

Comprender **embeddings semánticos y similitud** dentro de la evolución de la inteligencia artificial, implementar un experimento mínimo verificable y distinguir qué parte constituye evidencia frente a una afirmación todavía no comprobada.

📚 Resultados de aprendizaje

Al finalizar podrás:

1. Explicar embeddings semánticos y similitud usando los conceptos `embeddings`, `similitud`, `clustering`, `búsqueda`.
2. Ejecutar el laboratorio con una semilla explícita y revisar su contrato JSON.
3. Identificar al menos un supuesto, una limitación y un riesgo de aplicación.
4. Comparar el enfoque con la etapa anterior de la ruta de aprendizaje.
5. Producir una evidencia reproducible y una conclusión que no exceda los datos.

🧩 Conceptos centrales

`embeddings`, `similitud`, `clustering`, `búsqueda`

🗺️ Ubicación en el mapa de la IA

word2vec (Mikolov et al., 2013) demostró que un objetivo simple —predecir palabras vecinas— produce vectores donde la semántica es geometría: sinónimos cerca, analogías como aritmética. Esa idea es hoy la infraestructura invisible de la IA moderna: los embeddings alimentan la búsqueda semántica y el RAG (parte 08), los sistemas de recomendación, la detección de duplicados y el espacio compartido de CLIP (clase 069). Esta clase formaliza qué es un embedding, cómo se entrena, cómo se mide la similitud y qué sesgos hereda.

📖 Fundamentos

🧭 La hipótesis distribucional

"Conocerás una palabra por la compañía que mantiene" (Firth, 1957): palabras que aparecen en contextos similares tienen significados similares. Un **embedding semántico** materializa esta hipótesis: una función que asigna a cada texto (palabra, frase, documento) un vector denso en `R^d` tal que la proximidad geométrica aproxime la similitud de significado.

⚙️ word2vec: skip-gram con muestreo negativo

Skip-gram entrena cada palabra para predecir sus vecinas dentro de una ventana. Con **muestreo negativo**, el problema se vuelve una clasificación binaria barata:

```
Para cada par observado (centro c, contexto o):
  maximizar  log σ(v_o · v_c)                          # par real → similitud alta
  y para k palabras aleatorias n_i (negativos):
  maximizar  Σ log σ(−v_{n_i} · v_c)                   # pares falsos → similitud baja
```

Cada actualización acerca el vector de la palabra central al de su contexto real y lo aleja de contextos aleatorios. Tras ver miles de millones de pares, "perro" y "can" acaban cerca porque comparten vecinos ("ladra", "correa", "veterinario"). CBOW es la variante inversa (contexto → centro). El resultado famoso: `v(rey) − v(hombre) + v(mujer) ≈ v(reina)` — las direcciones del espacio capturan relaciones (género, capital-de, tiempo verbal) de forma aproximada.

📐 Similitud coseno

La métrica estándar compara direcciones, no magnitudes:

```
cos(u, v) = (u · v) / (‖u‖ ‖v‖)     ∈ [−1, 1]
```

1 = misma dirección, 0 = ortogonales, −1 = opuestos. Se prefiere sobre la distancia euclídea porque la norma de un embedding suele correlacionar con la frecuencia del término, no con su significado. Sobre vectores **normalizados** ($\|v\| = 1$), el coseno equivale al producto punto y la búsqueda del vecino más cercano se acelera con índices aproximados (ANN: HNSW, IVF — parte 08).

🧱 De palabras a frases: embeddings contextuales

word2vec asigna **un** vector por palabra: "banco" (asiento) y "banco" (entidad financiera) colapsan en el mismo punto. Los encoders tipo BERT producen embeddings **contextuales** (un vector por token *en su frase*), y modelos como Sentence-BERT se afinan con pares de frases para que el coseno entre vectores de frase mida directamente similitud semántica — la base de la búsqueda semántica y del RAG.

⚖️ Sesgo en embeddings

Los embeddings destilan las regularidades de su corpus, incluidas las indeseables. Bolukbasi et al. (2016) mostraron en word2vec entrenado sobre noticias: `v(programador) − v(hombre) + v(mujer) ≈ v(ama de casa)`. El sesgo no es un bug del algoritmo sino un reflejo del texto de entrenamiento, y se propaga silenciosamente a cualquier sistema construido encima (búsqueda de CV, moderación, recomendación). Las mitigaciones (proyección fuera del subespacio de género, contrapesos de datos) reducen métricas de sesgo específicas pero no lo eliminan: hay que **medirlo** en la tarea final.

🧮 Ejemplo trabajado

Tres embeddings 3D de juguete:

```
gato  = (1, 2, 2)      perro = (2, 1, 2)      coche = (2, 0, −1)

cos(gato, perro) = (1·2 + 2·1 + 2·2) / (‖gato‖ ‖perro‖)
                 = 8 / (3 · 3) = 0.889
  con ‖gato‖ = √(1+4+4) = 3,  ‖perro‖ = √(4+1+4) = 3

cos(gato, coche) = (1·2 + 2·0 + 2·(−1)) / (3 · 3) = 0 / 9 = 0.000
cos(perro, coche) = (4 + 0 − 2) / (3 · 3) = 2 / 9 = 0.222
```

`gato` y `perro` comparten dirección (0.889); `gato` y `coche` son ortogonales (0.0). Nota que `coche` tiene norma 3 igual que los otros: la magnitud no distingue nada aquí, todo el significado está en la **dirección**. Un buscador semántico que indexara estos tres vectores devolvería `perro` como vecino más cercano de `gato`.

📊 Propiedades y comparación

Método	Vector por	Contexto	Entrenamiento	Límite principal
TF-IDF (clase 065)	documento	No (bolsa de palabras)	Ninguno (conteos)	Sin sinonimia: "auto" ≠ "coche"
word2vec / GloVe	palabra (estático)	Ventana local / global	Auto-supervisado, barato	Polisemia colapsada
BERT (token)	token en su frase	Bidireccional completo	Preentrenamiento masivo	No optimizado para similitud de frases
Sentence-BERT / E5	frase o pasaje	Completo + fine-tune contrastivo	Pares de similitud	Dominio del fine-tune manda

⚠️ Errores conceptuales frecuentes

1. **"Coseno alto = mismo significado."** El coseno mide co-ocurrencia aprendida: antónimos ("subir"/"bajar") comparten contextos y suelen tener similitud alta. Cerca en el espacio ≠ intercambiables.
2. **"La aritmética de analogías es una propiedad exacta."** `rey − hombre + mujer` cae *cerca* de `reina` solo si se excluyen los términos de la consulta y en analogías frecuentes; es una regularidad aproximada, no un álgebra garantizada.
3. **"Los embeddings son neutrales porque son matemáticos."** Codifican los estereotipos estadísticos de su corpus (Bolukbasi 2016); usarlos en decisiones sobre personas sin auditoría es importar ese sesgo.

4. **"Un vector por palabra basta."** La polisemia rompe los embeddings estáticos: "banco" financiero y "banco" de plaza comparten vector. Para frases y documentos se necesitan embeddings contextuales afinados para similitud.
5. **"Embeddings de modelos distintos son comparables."** Cada modelo define su propio espacio: el coseno entre un vector de word2vec y uno de E5 no significa nada; en un índice vectorial no se pueden mezclar versiones de modelo.

🚀 Del aprendizaje a la operación

Un sistema de búsqueda semántica real añade: elección y **versionado** del modelo de embedding (cambiarlo obliga a reindexar todo), índices aproximados (HNSW) con su trade-off recall/latencia, evaluación de recuperación con consultas reales anotadas (recall@k, MRR), auditoría de sesgo cuando los resultados afectan a personas, y monitoreo de deriva de dominio: un embedding entrenado en texto general degrada en jerga médica o legal sin que ningún error explícito lo delate.

🧪 Laboratorio

```
python lab.py
```

El laboratorio llama a `ai_evolution.labs.run_lab("retrieval")`. Esta decisión evita una implementación divergente por clase: cada una conserva un entrypoint propio, pero los motores didácticos se prueban como una biblioteca común.

🔍 Evidencia esperada

- tipo de laboratorio y semilla;
- entradas o decisiones observables;
- resultado estructurado;
- lista `evidence` con hechos que pueden inspeccionarse;
- lista `limitations` que impide presentar la demo como producción.

📓 Notebooks

- 📓 `notebook.ipynb`: recorrido guiado con la materia resumida.
- ✍️ `notebook_student.ipynb`: ejercicios para resolver.
- ✅ `notebook_solution.ipynb`: solución de referencia explicada.

📝 Evaluación

Criterio	Peso
Comprensión conceptual	25 %
Ejecución reproducible	25 %
Interpretación basada en evidencia	25 %
Riesgos, límites y mejora propuesta	25 %

Consulta assessment.md para preguntas y criterio de aceptación.

⚠️ Errores comunes

Síntoma	Causa probable	Corrección
El código corre, pero no hay conclusión	Se confundió ejecución con aprendizaje	Explica qué demuestra y qué no demuestra
El resultado cambia sin explicación	No se registró semilla o configuración	Conserva semilla, versión y parámetros
Se promete uso real	Se extrapoló desde una demo educativa	Declara entorno, datos, límites y revisión humana
Se copia una métrica aislada	No existe baseline ni costo de error	Añade comparación y criterio de decisión

❓ Preguntas frecuentes

¿Debo usar una API comercial?
No. El núcleo funciona localmente. Las extensiones LIVE se documentan por separado.

¿El laboratorio representa una implementación industrial?
No por sí solo. Enseña el contrato y el patrón; producción exige integración, seguridad, observabilidad, pruebas y operación.

¿Dónde profundizo?
Revisa las especializaciones enlazadas en el README raíz y la ruta siguiente.

🔗 Referencias

- Jurafsky, D. y Martin, J. H. *Speech and Language Processing* (3e), cap. 6 (Vector Semantics and Embeddings) — web.stanford.edu/~jurafsky/slp3 — uso: desarrollo extendido del tema
- Mikolov, T. et al. (2013). "Efficient Estimation of Word Representations in Vector Space" (word2vec) — arXiv:1301.3781 — uso: fuente primaria del mecanismo estudiado
- Pennington, J., Socher, R. y Manning, C. (2014). "GloVe: Global Vectors for Word Representation" — nlp.stanford.edu/projects/glove — uso: referencia consultada en su fuente original
- Bolukbasi, T. et al. (2016). "Man is to Computer Programmer as Woman is to Homemaker? Debiasing Word Embeddings" — arXiv:1607.06520 — uso: fuente primaria del mecanismo estudiado
- Reimers, N. y Gurevych, I. (2019). "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks" — arXiv:1908.10084 — uso: fuente primaria del mecanismo estudiado
- Documentación oficial de Sentence-Transformers — sbert.net — uso: referencia consultada en su fuente original

---

📜 Papers que fundamentan esta clase

> Bloque generado por `python scripts/link_papers_to_classes.py`. La fuente es `papers/catalog/papers.json`.

Paper	Año	Qué desbloqueó	Miniatura
P05 · Estimación eficiente de representaciones de palabras en un espacio vectorial	2013	El significado distribucional se vuelve barato: vectores densos entrenables sobre miles de millones de palabras.	notebook
P23 · GloVe: vectores globales para representación de palabras	2014	Unifica las dos familias de embeddings: factorizar estadísticas globales de co-ocurrencia con la ventaja de los métodos predictivos.	notebook
P24 · Representaciones profundas de palabras dependientes del contexto	2018	Un vector por APARICIÓN y no por palabra: la polisemia deja de colapsar en un único punto del espacio.	notebook

Cada ficha explica el problema anterior, la matemática mínima, los límites y los errores de atribución más frecuentes. Para leerlas con método: cómo leer un paper de IA · anexos matemáticos.

📚 Bibliografía de apoyo

> Bloque generado por `python scripts/link_sources_to_classes.py`. Cada obra lleva su localizador verificado en `sources/bibliography.json`.

Los papers dicen **de dónde salió** el mecanismo. Estas obras lo **desarrollan** con el espacio que una clase no tiene: teoría completa, demostraciones y ejercicios.

Obra	Edición	Localizador	Papel en esta clase
Jurafsky, Daniel y Martin, James H. — *Speech and Language Processing*	2.ª (la 3.ª circula como borrador abierto sin ISBN) · 2009	ISBN 9780131873216 · web de la obra	citada en las referencias de esta clase · cap. 6 · obra de referencia de la parte 05

📝 Evaluación completa

❓ Preguntas

1. Define embeddings semánticos y similitud sin usar una marca o framework como definición.
2. Explica la relación entre embeddings, similitud, clustering, búsqueda.
3. Ejecuta `lab.py` dos veces con la misma semilla. ¿Qué debe conservarse?
4. Identifica una afirmación permitida y una afirmación exagerada sobre el resultado.
5. Propón una prueba negativa o un caso límite.

🏆 Reto verificable

Amplía el resultado del laboratorio con una clave `student_extension` que incluya:

- el supuesto que estás probando;
- una medición o comprobación;
- la conclusión;
- una limitación.

✅ Criterio de aceptación

- [ ] `lab.py` termina con código 0.
- [ ] El resultado contiene `kind`, `seed`, `evidence` y `limitations`.
- [ ] La extensión no modifica el comportamiento de otras clases.
- [ ] La interpretación referencia datos impresos por el laboratorio.
- [ ] Se declara al menos un riesgo o condición de no uso.

---

067 — Reconocimiento automático del habla

Parte: 05 — Lenguaje, visión, audio e IA multimodal
Nivel: avanzado · **Horas estimadas:** 6
Laboratorio: `perception` · **Estado:** `EXECUTABLE_CORE`

🎯 Propósito

Comprender **reconocimiento automático del habla** dentro de la evolución de la inteligencia artificial, implementar un experimento mínimo verificable y distinguir qué parte constituye evidencia frente a una afirmación todavía no comprobada.

📚 Resultados de aprendizaje

Al finalizar podrás:

1. Explicar reconocimiento automático del habla usando los conceptos `ASR`, `espectrograma`, `transcripción`, `WER`.
2. Ejecutar el laboratorio con una semilla explícita y revisar su contrato JSON.
3. Identificar al menos un supuesto, una limitación y un riesgo de aplicación.
4. Comparar el enfoque con la etapa anterior de la ruta de aprendizaje.
5. Producir una evidencia reproducible y una conclusión que no exceda los datos.

🧩 Conceptos centrales

`ASR`, `espectrograma`, `transcripción`, `WER`

🗺️ Ubicación en el mapa de la IA

El reconocimiento automático del habla (ASR) fue durante décadas el banco de pruebas del modelado secuencial probabilístico (HMM + modelos de mezclas), luego de las redes recurrentes con CTC (que ya viste en OCR, clase 063) y hoy de los transformers encoder-decoder entrenados a escala masiva (Whisper, 2022). El ASR convierte la modalidad audio en texto y con ello conecta todo lo anterior de esta parte con la voz: es la entrada de los asistentes conversacionales, del subtitulado accesible (clase 072) y el complemento exacto de la síntesis de voz (clase 068).

📖 Fundamentos

🎙️ Del aire a los números

El micrófono convierte presión sonora en una señal continua que se **muestrea** (16 000 muestras/s es el estándar en ASR) y **cuantiza** (16 bits). Un segundo de voz son 16 000 números: demasiado crudo y redundante para modelar directamente. El primer paso es extraer una representación tiempo-frecuencia.

📊 Espectrogramas y MFCC

1. **Ventanas:** la señal se corta en tramas de ~25 ms con salto (hop) de 10 ms — la voz es aproximadamente estacionaria a esa escala.
2. **FFT por trama:** energía en cada frecuencia → **espectrograma** (matriz tiempo × frecuencia).
3. **Escala mel:** los filtros se espacian según la percepción humana (más resolución en graves); con log de la energía → **log-mel espectrograma**, la entrada estándar de los modelos neuronales actuales (Whisper usa 80 bandas mel).
4. **MFCC:** una transformada coseno discreta sobre el log-mel decorrelaciona los coeficientes; los primeros 12–13 describen la envolvente espectral (el "timbre" de cada fonema). Eran la entrada estándar de la era HMM-GMM.

🔗 El problema de alineación y CTC

Un audio de 3 s son ~300 tramas, pero la transcripción tiene ~10 palabras: no se sabe qué trama corresponde a qué letra. **CTC** (Graves et al., 2006) resuelve la alineación igual que en OCR: la red emite una distribución por trama sobre el alfabeto más el blanco `∅`, y la pérdida suma la probabilidad de **todas** las alineaciones que colapsan a la transcripción correcta (algoritmo forward-backward, programación dinámica). CTC asume independencia condicional entre tramas, por eso suele combinarse con un modelo de lenguaje externo durante la decodificación (beam search).

🌐 Whisper: ASR como seq2seq a escala

Whisper (Radford et al., 2022) abandona CTC: un transformer encoder-decoder recibe el log-mel y **genera** el texto token a token, como una traducción audio→texto. Sus claves:

- Entrenado con **680 000 horas** de audio-texto recolectadas de la web (supervisión débil), multilingüe y multitarea (transcribir, traducir, detectar idioma) con tokens especiales de control.
- Robusto a acentos, ruido y dominios sin fine-tuning — la escala y diversidad del corpus sustituyen la adaptación manual.
- Costo: decodificación autoregresiva (más lenta que CTC) y **alucinaciones**: en silencios o música puede generar texto plausible que nadie dijo.

📏 WER: la métrica del ASR

```
WER = (S + D + I) / N
```

con S sustituciones, D borrados, I inserciones (alineación de Levenshtein a nivel de palabra) y N las palabras de la referencia. Ojo: WER puede superar el 100 % (inserciones en exceso). Un WER global esconde estructura: los sistemas comerciales muestran WER significativamente peor para acentos regionales, hablantes no nativos y voces infantiles — la equidad se mide **por subgrupo de hablantes**, no en promedio.

🧮 Ejemplo trabajado

Referencia (N = 6): `el modelo transcribe la voz humana` Hipótesis del ASR: `el modelos transcribe voz humana ya`

```
Alineación óptima:
el      modelo   transcribe  la  voz  humana  —
el      modelos  transcribe  —   voz  humana  ya
OK      S        OK          D   OK   OK      I

S = 1 (modelo→modelos), D = 1 (falta "la"), I = 1 (sobra "ya")
WER = (1 + 1 + 1) / 6 = 0.5  →  50 %
```

Nota cómo un error morfológico mínimo ("modelos") cuenta igual que inventar una palabra: WER es ciego a la gravedad semántica. Por eso en dominios críticos (medicina, justicia) se complementa con revisión de términos clave.

Tramas de un audio. Un clip de 2 s a 16 kHz con ventana de 25 ms y hop de 10 ms produce `1 + (2000 − 25)/10 ≈ 198` tramas; con 80 bandas mel, la entrada del modelo es una matriz de `198 × 80`.

📊 Propiedades y comparación

Era / sistema	Modelo acústico	Alineación	Datos típicos	Límite
HMM-GMM (1990s–2010s)	Mezclas gaussianas sobre MFCC	Estados HMM	100–1 000 h transcritas	Pipeline complejo, frágil
RNN + CTC (2013–)	Red recurrente fin-a-fin	CTC (todas las alineaciones)	1 000–10 000 h	Independencia condicional; necesita LM
Transformer seq2seq (Whisper, 2022)	Encoder-decoder sobre log-mel	Atención (implícita)	680 000 h (supervisión débil)	Lento (autoregresivo); alucina en silencio

⚠️ Errores conceptuales frecuentes

1. **"El ASR oye palabras."** El modelo ve energía tiempo-frecuencia; "oír" la palabra correcta depende tanto del modelo de lenguaje implícito como de la acústica — por eso inventa palabras plausibles ante audio degradado.
2. **"WER 5 % significa que el 95 % de las transcripciones son correctas."** WER es una tasa de error por palabra, no por transcripción: con frases de 20 palabras y errores independientes, casi dos tercios de las frases tendrían algún error.
3. **"Un buen WER promedio implica un sistema justo."** Los errores se concentran en acentos y voces subrepresentadas en el entrenamiento; sin desglose por subgrupo, el promedio oculta a quién falla.

4. **"Whisper no puede equivocarse si el audio está en silencio."** Al contrario: la decodificación autoregresiva sobre silencio o música es el caso típico de alucinación — texto fluido que nadie pronunció.
5. **"Más horas de audio siempre arreglan el dominio."** Si tu dominio (jerga médica, radio de aviación) no está en el corpus, la escala general no lo cubre: hace falta adaptación con datos del dominio y léxico específico.

🚀 Del aprendizaje a la operación

Un ASR en producción añade: detección de actividad de voz (VAD) para no transcribir silencio, streaming con latencia parcial (transcripción incremental), diarización (¿quién habla?), puntuación y normalización inversa de números/fechas, métricas por subgrupo de hablantes y por término crítico, y un umbral de confianza que derive a transcripción humana cuando el costo del error lo exija (actas legales, indicaciones médicas).

🧪 Laboratorio

```
python lab.py
```

El laboratorio llama a `ai_evolution.labs.run_lab("perception")`. Esta decisión evita una implementación divergente por clase: cada una conserva un entrypoint propio, pero los motores didácticos se prueban como una biblioteca común.

🔍 Evidencia esperada

- tipo de laboratorio y semilla;
- entradas o decisiones observables;
- resultado estructurado;
- lista `evidence` con hechos que pueden inspeccionarse;
- lista `limitations` que impide presentar la demo como producción.

📓 Notebooks

- 📓 `notebook.ipynb`: recorrido guiado con la materia resumida.
- ✍️ `notebook_student.ipynb`: ejercicios para resolver.
- ✅ `notebook_solution.ipynb`: solución de referencia explicada.

📝 Evaluación

Criterio	Peso
Comprensión conceptual	25 %
Ejecución reproducible	25 %
Interpretación basada en evidencia	25 %
Riesgos, límites y mejora propuesta	25 %

Consulta assessment.md para preguntas y criterio de aceptación.

⚠️ Errores comunes

Síntoma	Causa probable	Corrección
El código corre, pero no hay conclusión	Se confundió ejecución con aprendizaje	Explica qué demuestra y qué no demuestra
El resultado cambia sin explicación	No se registró semilla o configuración	Conserva semilla, versión y parámetros
Se promete uso real	Se extrapoló desde una demo educativa	Declara entorno, datos, límites y revisión humana
Se copia una métrica aislada	No existe baseline ni costo de error	Añade comparación y criterio de decisión

❓ Preguntas frecuentes

¿Debo usar una API comercial?
No. El núcleo funciona localmente. Las extensiones LIVE se documentan por separado.

¿El laboratorio representa una implementación industrial?
No por sí solo. Enseña el contrato y el patrón; producción exige integración, seguridad, observabilidad, pruebas y operación.

¿Dónde profundizo?
Revisa las especializaciones enlazadas en el README raíz y la ruta siguiente.

🔗 Referencias

- Jurafsky, D. y Martin, J. H. *Speech and Language Processing* (3e), cap. de Automatic Speech Recognition — web.stanford.edu/~jurafsky/slp3 — uso: desarrollo extendido del tema
- Graves, A. et al. (2006). "Connectionist Temporal Classification: Labelling Unsegmented Sequence Data with Recurrent Neural Networks" (ICML 2006) — cs.toronto.edu/~graves/icml_2006.pdf — uso: referencia consultada en su fuente original
- Radford, A. et al. (2022). "Robust Speech Recognition via Large-Scale Weak Supervision" (Whisper) — arXiv:2212.04356 — uso: fuente primaria del mecanismo estudiado
- Repositorio oficial de Whisper (OpenAI) — github.com/openai/whisper — uso: referencia consultada en su fuente original
- Documentación de librosa (análisis de audio: espectrogramas, MFCC) — librosa.org/doc — uso: referencia consultada en su fuente original
- Mozilla Common Voice (corpus abierto multilingüe de voz) — commonvoice.mozilla.org — uso: referencia consultada en su fuente original

---

📜 Papers que fundamentan esta clase

> Bloque generado por `python scripts/link_papers_to_classes.py`. La fuente es `papers/catalog/papers.json`.

Paper	Año	Qué desbloqueó	Miniatura
P06 · Aprendizaje de secuencia a secuencia con redes neuronales	2014	Una única red aprende a mapear secuencias de longitud variable a secuencias de longitud variable, de extremo a extremo.	notebook

Cada ficha explica el problema anterior, la matemática mínima, los límites y los errores de atribución más frecuentes. Para leerlas con método: cómo leer un paper de IA · anexos matemáticos.

📚 Bibliografía de apoyo

> Bloque generado por `python scripts/link_sources_to_classes.py`. Cada obra lleva su localizador verificado en `sources/bibliography.json`.

Los papers dicen **de dónde salió** el mecanismo. Estas obras lo **desarrollan** con el espacio que una clase no tiene: teoría completa, demostraciones y ejercicios.

Obra	Edición	Localizador	Papel en esta clase
Jurafsky, Daniel y Martin, James H. — *Speech and Language Processing*	2.ª (la 3.ª circula como borrador abierto sin ISBN) · 2009	ISBN 9780131873216 · web de la obra	citada en las referencias de esta clase · obra de referencia de la parte 05

📝 Evaluación completa

❓ Preguntas

1. Define reconocimiento automático del habla sin usar una marca o framework como definición.
2. Explica la relación entre ASR, espectrograma, transcripción, WER.
3. Ejecuta `lab.py` dos veces con la misma semilla. ¿Qué debe conservarse?
4. Identifica una afirmación permitida y una afirmación exagerada sobre el resultado.
5. Propón una prueba negativa o un caso límite.

🏆 Reto verificable

Amplía el resultado del laboratorio con una clave `student_extension` que incluya:

- el supuesto que estás probando;
- una medición o comprobación;
- la conclusión;

- una limitación.

✅ Criterio de aceptación

- [ ] `lab.py` termina con código 0.
- [ ] El resultado contiene `kind`, `seed`, `evidence` y `limitations`.
- [ ] La extensión no modifica el comportamiento de otras clases.
- [ ] La interpretación referencia datos impresos por el laboratorio.
- [ ] Se declara al menos un riesgo o condición de no uso.

---

068 — Síntesis de voz y clonación responsable

Parte: 05 — Lenguaje, visión, audio e IA multimodal
Nivel: avanzado · **Horas estimadas:** 6
Laboratorio: `generation` · **Estado:** `EXECUTABLE_CORE`

🎯 Propósito

Comprender **síntesis de voz y clonación responsable** dentro de la evolución de la inteligencia artificial, implementar un experimento mínimo verificable y distinguir qué parte constituye evidencia frente a una afirmación todavía no comprobada.

📚 Resultados de aprendizaje

Al finalizar podrás:

1. Explicar síntesis de voz y clonación responsable usando los conceptos `TTS`, `voz`, `consentimiento`, `watermark`.
2. Ejecutar el laboratorio con una semilla explícita y revisar su contrato JSON.
3. Identificar al menos un supuesto, una limitación y un riesgo de aplicación.
4. Comparar el enfoque con la etapa anterior de la ruta de aprendizaje.
5. Producir una evidencia reproducible y una conclusión que no exceda los datos.

🧩 Conceptos centrales

`TTS`, `voz`, `consentimiento`, `watermark`

🗺️ Ubicación en el mapa de la IA

La síntesis de voz (TTS, *text-to-speech*) es la operación inversa del ASR de la clase anterior: convierte texto en audio. Pasó de la concatenación de fragmentos grabados a la generación neuronal directa de la onda con WaveNet (2016), que demostró que un modelo autoregresivo podía producir voz casi indistinguible de la humana. Esa capacidad habilita asistentes conversacionales y tecnología accesible (clase 072), pero también la **clonación de voz** con segundos de audio, lo que convierte el consentimiento, el marcado (watermark) y la detección de audio sintético en parte inseparable de la técnica.

📖 Fundamentos

🔤 Frontend de texto: normalización y G2P

El texto crudo no se pronuncia tal cual. El frontend aplica:

1. **Normalización:** expandir números, fechas, siglas y abreviaturas (`"Dr. Ruiz, 3/5/2026"` → `"doctor Ruiz, tres de mayo de dos mil veintiséis"`). Es dependiente del idioma y una fuente clásica de errores audibles.

2. **G2P (grapheme-to-phoneme):** convertir letras en fonemas. En español la relación es bastante regular; en inglés no (`though`, `tough`, `through`). Los sistemas modernos pueden operar sobre caracteres, pero el fonema sigue dando control fino de pronunciación.

🎼 Modelo acústico: texto → espectrograma (Tacotron 2)

Un seq2seq con atención (Tacotron 2, 2018) recibe la secuencia de fonemas/caracteres y **predice el log-mel espectrograma** trama a trama (80 bandas, hop ~12,5 ms), más un *stop token* que indica cuándo terminar. La atención aprende el alineamiento texto↔tiempo: cada trama de audio "mira" a los fonemas que está pronunciando. La prosodia (entonación, pausas, ritmo) emerge del corpus de entrenamiento: un solo hablante profesional con ~24 h de audio limpio en el caso clásico.

🔊 Vocoder: espectrograma → onda

El mel descarta la fase: no se puede invertir directamente a audio. El **vocoder** genera la onda condicionada en el mel:

- **WaveNet** (van den Oord et al., 2016) modela la onda muestra a muestra:

$$p(x) = \prod_t p(x_t \mid x_1 \dots x_{t-1})$$

con **convoluciones causales dilatadas** (dilación 1, 2, 4, …, 512) que duplican el campo receptivo por capa. Calidad excelente, pero generación secuencial: un segundo de audio a 16 kHz exige 16 000 pasos uno tras otro. - **Vocoders paralelos** (HiFi-GAN y familia): redes generativas adversarias que producen toda la onda de una pasada — cientos de veces más rápidos, calidad comparable, y son el estándar práctico actual.

🗣️ Clonación de voz

La clonación desacopla **qué se dice** de **quién lo dice**. Un *speaker encoder* (entrenado para verificación de hablante) comprime unos segundos de voz en un vector de identidad (*d-vector* / *speaker embedding*); el modelo acústico se condiciona en ese vector (SV2TTS, 2018). Consecuencia técnica y ética a la vez: **bastan segundos de audio público para clonar una voz**, sin que el hablante participe ni lo sepa.

🛡️ Consentimiento, watermarking y detección

- **Consentimiento:** explícito, informado, documentado y con alcance definido (qué frases, en qué contextos, por cuánto tiempo, con derecho a revocación). Clonar la voz de alguien sin él es suplantación, no "una demo".
- **Watermarking de audio:** incrustar en la señal sintética un patrón imperceptible al oído pero detectable por un algoritmo, para poder responder después "¿este audio lo generó mi sistema?". No es infalible: recompresión, re-grabación con micrófono y filtrado pueden degradar la marca.
- **Detección de audio sintético:** clasificadores que buscan artefactos del vocoder. Es una carrera armamentista: cada generación de modelos borra los artefactos que detectaba la anterior; la defensa no puede descansar solo en el detector.

📏 Evaluación: MOS e inteligibilidad

- **MOS (Mean Opinion Score):** oyentes puntúan naturalidad de 1 a 5; se reporta la media (y debería reportarse el intervalo). Es subjetivo, caro y sensible al panel de oyentes.

- **Inteligibilidad objetiva (proxy):** pasar el audio sintético por un ASR y medir el WER contra el texto original — barato y automatizable, pero mide otra cosa que la naturalidad.

🧮 Ejemplo trabajado

Campo receptivo de WaveNet. Una pila de 10 convoluciones causales con kernel 2 y dilaciones 1, 2, 4, …, 512:

```
campo receptivo = 1 + Σ dilaciones = 1 + (1+2+4+…+512) = 1 + 1023 = 1024 muestras
a 16 kHz → 1024 / 16000 = 64 ms de contexto
con 3 pilas apiladas: 1 + 3·1023 = 3070 muestras ≈ 192 ms
```

Costo autoregresivo. Generar 2 s de audio a 16 kHz = 32 000 pasos secuenciales (cada muestra espera a la anterior). Un vocoder paralelo produce las 32 000 muestras en una pasada: esa es la diferencia entre demo de laboratorio y asistente en tiempo real.

MOS con los mismos promedios. Sistema A: [4, 4, 5, 3, 4] → media 4,0, desvío 0,63. Sistema B: [5, 5, 5, 1, 4] → media 4,0, desvío 1,55. Igual MOS medio, experiencias muy distintas: B suena excelente casi siempre y falla feo a veces. La media sola no decide.

📊 Propiedades y comparación

Enfoque	Cómo genera	Calidad típica	Velocidad	Límite principal
Concatenativo (1990s–2000s)	Pega fragmentos grabados	Media, con "costuras"	Rápida	Rígido: solo lo grabado
Paramétrico HMM (2000s)	Estadísticas de vocoder clásico	Inteligible pero "robótica"	Rápida	Voz apagada, sobre-suavizada
Neuronal autoregresivo (WaveNet, 2016)	Muestra a muestra	Casi humana	Muy lenta	16 000 pasos/segundo de audio
Neuronal paralelo (HiFi-GAN, 2020)	Onda completa en una pasada	Casi humana	Tiempo real	Entrenamiento adversario delicado

⚠️ Errores conceptuales frecuentes

1. **"El TTS lee letras."** Lee texto normalizado y fonemas: sin frontend, "2026" o "Dr." se pronuncian mal. Gran parte de los errores audibles nacen antes de la red neuronal.

2. **"El espectrograma ya es el audio."** El mel descarta la fase; sin vocoder no hay onda. Modelo acústico y vocoder son dos problemas distintos con fallos distintos.
3. **"Clonar una voz exige horas de grabación."** Con speaker embeddings bastan segundos de audio público. Subestimar esto es subestimar el riesgo de suplantación.
4. **"El watermark resuelve el problema de los deepfakes."** Es una capa útil pero degradable (re-grabación, compresión) y solo cubre a los generadores que lo implementan. La defensa real combina marca, detección, procedencia y norma.
5. **"MOS 4,5 significa sistema terminado."** El MOS es subjetivo, depende del panel y del texto evaluado, y no mide robustez ante texto fuera de dominio ni errores de normalización.

🚀 Del aprendizaje a la operación

Un TTS operativo añade: síntesis en *streaming* con latencia de primeras muestras < 300 ms, control de prosodia y pronunciación (SSML, léxicos por dominio), registro auditable de consentimientos con alcance y revocación, watermarking activado por defecto más un canal de verificación, y monitoreo de abuso (volumen anómalo de clonaciones, frases sensibles). Nada de eso está en esta demo educativa.

🧪 Laboratorio

```
python lab.py
```

El laboratorio llama a `ai_evolution.labs.run_lab("generation")`. Esta decisión evita una implementación divergente por clase: cada una conserva un entrypoint propio, pero los motores didácticos se prueban como una biblioteca común.

🔍 Evidencia esperada

- tipo de laboratorio y semilla;
- entradas o decisiones observables;
- resultado estructurado;
- lista `evidence` con hechos que pueden inspeccionarse;
- lista `limitations` que impide presentar la demo como producción.

📓 Notebooks

- 📓 `notebook.ipynb`: recorrido guiado con la materia resumida.
- ✍️ `notebook_student.ipynb`: ejercicios para resolver.
- ✅ `notebook_solution.ipynb`: solución de referencia explicada.

📝 Evaluación

Criterio	Peso
Comprensión conceptual	25 %
Ejecución reproducible	25 %
Interpretación basada en evidencia	25 %
Riesgos, límites y mejora propuesta	25 %

Consulta assessment.md para preguntas y criterio de aceptación.

⚠️ Errores comunes

Síntoma	Causa probable	Corrección
El código corre, pero no hay conclusión	Se confundió ejecución con aprendizaje	Explica qué demuestra y qué no demuestra
El resultado cambia sin explicación	No se registró semilla o configuración	Conserva semilla, versión y parámetros
Se promete uso real	Se extrapoló desde una demo educativa	Declara entorno, datos, límites y revisión humana
Se copia una métrica aislada	No existe baseline ni costo de error	Añade comparación y criterio de decisión

❓ Preguntas frecuentes

¿Debo usar una API comercial?
No. El núcleo funciona localmente. Las extensiones LIVE se documentan por separado.

¿El laboratorio representa una implementación industrial?
No por sí solo. Enseña el contrato y el patrón; producción exige integración, seguridad, observabilidad, pruebas y operación.

¿Dónde profundizo?
Revisa las especializaciones enlazadas en el README raíz y la ruta siguiente.

🔗 Referencias

- van den Oord, A. et al. (2016). "WaveNet: A Generative Model for Raw Audio" — arXiv:1609.03499 — uso: fuente primaria del mecanismo estudiado
- Shen, J. et al. (2018). "Natural TTS Synthesis by Conditioning WaveNet on Mel Spectrogram Predictions" (Tacotron 2) — arXiv:1712.05884 — uso: fuente primaria del mecanismo estudiado
- Jia, Y. et al. (2018). "Transfer Learning from Speaker Verification to Multispeaker Text-To-Speech Synthesis" (SV2TTS) — arXiv:1806.04558 — uso: fuente primaria del mecanismo estudiado
- Jurafsky, D. y Martin, J. H. *Speech and Language Processing* (3e), cap. de síntesis de voz — web.stanford.edu/~jurafsky/slp3 — uso: desarrollo extendido del tema
- W3C. *Speech Synthesis Markup Language (SSML) 1.1* — w3.org/TR/speech-synthesis11 — uso: marco normativo de referencia

- Mozilla Common Voice (corpus abierto de voz con licencia y consentimiento explícitos) — commonvoice.mozilla.org — uso: referencia consultada en su fuente original

---

📜 Papers que fundamentan esta clase

> Bloque generado por `python scripts/link_papers_to_classes.py`. La fuente es `papers/catalog/papers.json`.

Paper	Año	Qué desbloqueó	Miniatura
P119 · WaveNet: un modelo generativo de audio en crudo	2016	Genera la forma de onda muestra a muestra con convoluciones causales dilatadas, y cierra la brecha de naturalidad que arrastraba la síntesis de voz.	notebook
P122 · Síntesis de voz natural condicionando WaveNet con espectrogramas mel predichos	2018	Parte la síntesis en dos etapas con el espectrograma mel como interfaz, y alcanza naturalidad indistinguible de una grabación en la escala de opinión media.	notebook

Cada ficha explica el problema anterior, la matemática mínima, los límites y los errores de atribución más frecuentes. Para leerlas con método: cómo leer un paper de IA · anexos matemáticos.

📚 Bibliografía de apoyo

> Bloque generado por `python scripts/link_sources_to_classes.py`. Cada obra lleva su localizador verificado en `sources/bibliography.json`.

Los papers dicen **de dónde salió** el mecanismo. Estas obras lo **desarrollan** con el espacio que una clase no tiene: teoría completa, demostraciones y ejercicios.

Obra	Edición	Localizador	Papel en esta clase
Jurafsky, Daniel y Martin, James H. — *Speech and Language Processing*	2.ª (la 3.ª circula como borrador abierto sin ISBN) · 2009	ISBN 9780131873216 · web de la obra	citada en las referencias de esta clase · obra de referencia de la parte 05

Normas y documentación oficial que aplica esta clase: w3.org/TR/speech-synthesis11

📝 Evaluación completa

❓ Preguntas

1. Define síntesis de voz y clonación responsable sin usar una marca o framework como definición.
2. Explica la relación entre TTS, voz, consentimiento, watermark.

3. Ejecuta `lab.py` dos veces con la misma semilla. ¿Qué debe conservarse?
4. Identifica una afirmación permitida y una afirmación exagerada sobre el resultado.
5. Propón una prueba negativa o un caso límite.

🏆 Reto verificable

Amplía el resultado del laboratorio con una clave `student_extension` que incluya:

- el supuesto que estás probando;
- una medición o comprobación;
- la conclusión;
- una limitación.

✅ Criterio de aceptación

- [ ] `lab.py` termina con código 0.
- [ ] El resultado contiene `kind`, `seed`, `evidence` y `limitations`.
- [ ] La extensión no modifica el comportamiento de otras clases.
- [ ] La interpretación referencia datos impresos por el laboratorio.
- [ ] Se declara al menos un riesgo o condición de no uso.

---

069 — Modelos visión-lenguaje

Parte: 05 — Lenguaje, visión, audio e IA multimodal
Nivel: avanzado · **Horas estimadas:** 6
Laboratorio: `attention` · **Estado:** `EXECUTABLE_CORE`

🎯 Propósito

Comprender **modelos visión-lenguaje** dentro de la evolución de la inteligencia artificial, implementar un experimento mínimo verificable y distinguir qué parte constituye evidencia frente a una afirmación todavía no comprobada.

📚 Resultados de aprendizaje

Al finalizar podrás:

1. Explicar modelos visión-lenguaje usando los conceptos `CLIP`, `VLM`, `alineamiento`, `grounding`.
2. Ejecutar el laboratorio con una semilla explícita y revisar su contrato JSON.
3. Identificar al menos un supuesto, una limitación y un riesgo de aplicación.
4. Comparar el enfoque con la etapa anterior de la ruta de aprendizaje.
5. Producir una evidencia reproducible y una conclusión que no exceda los datos.

🧩 Conceptos centrales

`CLIP`, `VLM`, `alineamiento`, `grounding`

🗺️ Ubicación en el mapa de la IA

Hasta aquí visión (clases 061-063) y lenguaje (064-066) vivían en espacios separados. CLIP (2021) los unió: entrenó un codificador de imágenes y uno de texto para que imagen y descripción correcta caigan **cerca en el mismo espacio vectorial**, usando 400 millones de pares imagen-texto de la web. Ese alineamiento habilita clasificación *zero-shot* sin reentrenar, y sus codificadores se volvieron piezas estándar de los generadores de imágenes y de los LLM multimodales. Es el puente directo hacia la fusión multimodal (clase 070).

📖 Fundamentos

🧲 Doble codificador y aprendizaje contrastivo

CLIP entrena dos redes en paralelo: un codificador de imágenes `f(imagen)` (ResNet o ViT) y un codificador de texto `g(texto)` (transformer). Ambos proyectan a un espacio común y se normalizan a longitud 1, de modo que la **similitud coseno** es un producto punto.

Con un lote de N pares (imagen, texto), se calcula la matriz N×N de similitudes. El objetivo contrastivo (InfoNCE) empuja la diagonal (pares verdaderos) hacia arriba y el resto hacia abajo, en ambas direcciones:

```
s_ij = f(img_i) · g(txt_j) / τ          (τ = temperatura, aprendida)

L = ½ [ CE por filas (imagen → texto correcto)
      + CE por columnas (texto → imagen correcta) ]
```

La **temperatura τ** escala las similitudes antes del softmax: τ pequeña produce distribuciones afiladas (castiga fuerte al segundo mejor), τ grande las suaviza. No hay etiquetas de clase: la supervisión es "este texto acompañaba a esta imagen en la web".

🎯 Clasificación zero-shot

Para clasificar sin entrenar nada nuevo:

1. Convertir cada clase en una frase-plantilla: `"una foto de un {perro}"`.
2. Codificar las frases con `g` → un embedding por clase.
3. Codificar la imagen con `f` y elegir la clase de mayor similitud coseno.

El *prompt* importa: "a photo of a dog" rinde mejor que "dog" porque se parece más a los pies de foto vistos en el entrenamiento; promediar varias plantillas (*prompt ensembling*) mejora aún más. Zero-shot no significa "sin datos": significa que los 400 M de pares del preentrenamiento ya cubrieron el concepto.

❓ VQA y grounding

- **VQA (Visual Question Answering):** responder preguntas en lenguaje natural sobre una imagen ("¿cuántas tazas hay?"). Exige combinar percepción, lenguaje y a veces conteo o razonamiento espacial — más que similitud global.
- **Grounding:** anclar palabras a regiones concretas de la imagen ("la taza *roja*" → esa caja). CLIP produce un embedding **global** por imagen: sabe *qué* hay, no *dónde*; el grounding fino requiere arquitecturas con atención espacial o detección (clase 062).
- Los VLM generativos (Flamingo, LLaVA) conectan un codificador visual con un LLM que **genera** texto condicionado en la imagen: eso permite VQA y descripción libres, con el costo de heredar las alucinaciones del LLM.

🌎 Datos, sesgos y fallos característicos

El par imagen-texto de la web trae sus sesgos: asociaciones culturales, estereotipos y desbalance de idiomas quedan impresos en el espacio compartido. Fallos conocidos de CLIP: comportamiento de "bolsa de palabras" (ordena mal relaciones como agente/paciente), debilidad en conteo y en relaciones espaciales, y **ataques tipográficos**: un papel con la palabra "iPod" pegado a una manzana desplaza el embedding hacia "iPod".

🧮 Ejemplo trabajado

Espacio compartido de dimensión 3, todo ya normalizado. Una imagen y tres textos:

```
img          = (0.8, 0.6, 0.0)
t_perro      = (1, 0, 0)     t_gato = (0, 1, 0)     t_avión = (0, 0, 1)

Similitudes coseno (producto punto):
s(img, t_perro) = 0.8      s(img, t_gato) = 0.6      s(img, t_avión) = 0.0

Softmax con temperatura τ = 0.5 → logits (1.6, 1.2, 0.0):
exp: (4.95, 3.32, 1.00)  suma = 9.27
p = (0.53, 0.36, 0.11)   → predicción zero-shot: "perro"

Con τ = 0.1 → logits (8, 6, 0): p ≈ (0.88, 0.12, 0.00) — misma decisión, mucha más
confianza aparente. La temperatura no cambia el ranking, cambia la calibración.
```

📊 Propiedades y comparación

Enfoque	Supervisión	¿Clases nuevas sin reentrenar?	¿Localiza?	Límite principal
Clasificador supervisado (061)	Etiquetas por clase	No	No	Congelado en sus clases
Detector (062)	Cajas etiquetadas	No	Sí	Anotación costosa
CLIP zero-shot (2021)	400 M pares web	Sí (vía prompt)	No	Bolsa de palabras, conteo
VLM generativo (Flamingo/LLaVA)	Pares + instrucciones	Sí (pregunta libre)	Parcial	Alucina detalles

⚠️ Errores conceptuales frecuentes

1. **"CLIP entiende la frase."** Se comporta en gran medida como bolsa de palabras: "un perro persigue a un gato" y "un gato persigue a un perro" caen casi en el mismo punto del espacio. El alineamiento es de conceptos, no de sintaxis.
2. **"Zero-shot = aprender sin datos."** Hubo 400 millones de pares de entrenamiento; lo que no hay es *fine-tuning* por tarea. Si tu dominio (radiografías, microscopía) no estaba en la web, el zero-shot se degrada fuerte.
3. **"Mayor similitud = verdad."** La similitud es relativa al conjunto de prompts que tú escribiste; si la clase verdadera no está entre las opciones, el argmax elige igual un ganador equivocado con confianza.
4. **"CLIP sabe dónde está el objeto."** El embedding es global; para cajas o máscaras hacen falta arquitecturas de detección/segmentación o grounding explícito.

5. **"El espacio compartido es neutral."** Hereda los sesgos y estereotipos del texto web que lo entrenó, y es vulnerable a texto adversario dentro de la propia imagen.

🚀 Del aprendizaje a la operación

Un VLM en producción exige: evaluación con un conjunto propio del dominio (no confiar en el zero-shot reportado en benchmarks), diseño y versionado de prompts de clase, calibración de umbrales de similitud para poder decir "ninguna de las anteriores", pruebas de robustez ante texto adversario en imagen, y auditoría de sesgos sobre subgrupos representativos antes de exponer decisiones a usuarios.

🧪 Laboratorio

```
python lab.py
```

El laboratorio llama a `ai_evolution.labs.run_lab("attention")`. Esta decisión evita una implementación divergente por clase: cada una conserva un entrypoint propio, pero los motores didácticos se prueban como una biblioteca común.

🔍 Evidencia esperada

- tipo de laboratorio y semilla;
- entradas o decisiones observables;
- resultado estructurado;
- lista `evidence` con hechos que pueden inspeccionarse;
- lista `limitations` que impide presentar la demo como producción.

📓 Notebooks

- 📓 `notebook.ipynb`: recorrido guiado con la materia resumida.
- ✍️ `notebook_student.ipynb`: ejercicios para resolver.
- ✅ `notebook_solution.ipynb`: solución de referencia explicada.

📝 Evaluación

Criterio	Peso
Comprensión conceptual	25 %
Ejecución reproducible	25 %
Interpretación basada en evidencia	25 %
Riesgos, límites y mejora propuesta	25 %

Consulta assessment.md para preguntas y criterio de aceptación.

⚠️ Errores comunes

Síntoma	Causa probable	Corrección
El código corre, pero no hay conclusión	Se confundió ejecución con aprendizaje	Explica qué demuestra y qué no demuestra
El resultado cambia sin explicación	No se registró semilla o configuración	Conserva semilla, versión y parámetros
Se promete uso real	Se extrapoló desde una demo educativa	Declara entorno, datos, límites y revisión humana
Se copia una métrica aislada	No existe baseline ni costo de error	Añade comparación y criterio de decisión

❓ Preguntas frecuentes

¿Debo usar una API comercial?
No. El núcleo funciona localmente. Las extensiones LIVE se documentan por separado.

¿El laboratorio representa una implementación industrial?
No por sí solo. Enseña el contrato y el patrón; producción exige integración, seguridad, observabilidad, pruebas y operación.

¿Dónde profundizo?
Revisa las especializaciones enlazadas en el README raíz y la ruta siguiente.

🔗 Referencias

- Radford, A. et al. (2021). "Learning Transferable Visual Models From Natural Language Supervision" (CLIP) — arXiv:2103.00020 — uso: fuente primaria del mecanismo estudiado
- van den Oord, A. et al. (2018). "Representation Learning with Contrastive Predictive Coding" (InfoNCE) — arXiv:1807.03748 — uso: fuente primaria del mecanismo estudiado
- Antol, S. et al. (2015). "VQA: Visual Question Answering" — arXiv:1505.00468 — uso: fuente primaria del mecanismo estudiado
- Alayrac, J.-B. et al. (2022). "Flamingo: a Visual Language Model for Few-Shot Learning" — arXiv:2204.14198 — uso: fuente primaria del mecanismo estudiado
- Repositorio oficial de CLIP (OpenAI) — github.com/openai/CLIP — uso: referencia consultada en su fuente original
- Szeliski, R. *Computer Vision: Algorithms and Applications* (2e) — szeliski.org/Book — uso: desarrollo extendido del tema

---

📜 Papers que fundamentan esta clase

> Bloque generado por `python scripts/link_papers_to_classes.py`. La fuente es `papers/catalog/papers.json`.

Paper	Año	Qué desbloqueó	Miniatura
P18 · Aprender modelos visuales transferibles con supervisión de lenguaje natural	2021	El texto se convierte en la etiqueta: un solo modelo clasifica categorías que nadie anotó, describiéndolas con palabras.	notebook
P46 · Una imagen vale 16x16 palabras: Transformers para reconocimiento de imágenes a escala	2020	Trata la imagen como una secuencia de parches y aplica un Transformer puro: la convolución deja de ser imprescindible en visión.	notebook

Cada ficha explica el problema anterior, la matemática mínima, los límites y los errores de atribución más frecuentes. Para leerlas con método: cómo leer un paper de IA · anexos matemáticos.

📚 Bibliografía de apoyo

> Bloque generado por `python scripts/link_sources_to_classes.py`. Cada obra lleva su localizador verificado en `sources/bibliography.json`.

Los papers dicen **de dónde salió** el mecanismo. Estas obras lo **desarrollan** con el espacio que una clase no tiene: teoría completa, demostraciones y ejercicios.

Obra	Edición	Localizador	Papel en esta clase
Szeliski, Richard — *Computer Vision: Algorithms and Applications*	2.ª · 2022	web de la obra · *pendiente de confirmar en su catálogo*	citada en las referencias de esta clase
Jurafsky, Daniel y Martin, James H. — *Speech and Language Processing*	2.ª (la 3.ª circula como borrador abierto sin ISBN) · 2009	ISBN 9780131873216 · web de la obra	obra de referencia de la parte 05 · lenguaje y habla

📝 Evaluación completa

❓ Preguntas

1. Define modelos visión-lenguaje sin usar una marca o framework como definición.
2. Explica la relación entre CLIP, VLM, alineamiento, grounding.
3. Ejecuta `lab.py` dos veces con la misma semilla. ¿Qué debe conservarse?
4. Identifica una afirmación permitida y una afirmación exagerada sobre el resultado.
5. Propón una prueba negativa o un caso límite.

🏆 Reto verificable

Amplía el resultado del laboratorio con una clave `student_extension` que incluya:

- el supuesto que estás probando;
- una medición o comprobación;

- la conclusión;
- una limitación.

✅ Criterio de aceptación

- [ ] `lab.py` termina con código 0.
- [ ] El resultado contiene `kind`, `seed`, `evidence` y `limitations`.
- [ ] La extensión no modifica el comportamiento de otras clases.
- [ ] La interpretación referencia datos impresos por el laboratorio.
- [ ] Se declara al menos un riesgo o condición de no uso.

---

070 — Fusión multimodal y representación conjunta

Parte: 05 — Lenguaje, visión, audio e IA multimodal
Nivel: avanzado · **Horas estimadas:** 6
Laboratorio: `attention` · **Estado:** `EXECUTABLE_CORE`

🎯 Propósito

Comprender **fusión multimodal y representación conjunta** dentro de la evolución de la inteligencia artificial, implementar un experimento mínimo verificable y distinguir qué parte constituye evidencia frente a una afirmación todavía no comprobada.

📚 Resultados de aprendizaje

Al finalizar podrás:

1. Explicar fusión multimodal y representación conjunta usando los conceptos `fusión`, `cross-attention`, `modalidades`, `alineamiento`.
2. Ejecutar el laboratorio con una semilla explícita y revisar su contrato JSON.
3. Identificar al menos un supuesto, una limitación y un riesgo de aplicación.
4. Comparar el enfoque con la etapa anterior de la ruta de aprendizaje.
5. Producir una evidencia reproducible y una conclusión que no exceda los datos.

🧩 Conceptos centrales

`fusión`, `cross-attention`, `modalidades`, `alineamiento`

🗺️ Ubicación en el mapa de la IA

Las clases anteriores produjeron representaciones por modalidad (visión, texto, audio) y CLIP mostró cómo **alinear** dos de ellas en un espacio común. La fusión multimodal es el paso siguiente: decidir *dónde y cómo* combinar las modalidades dentro de un modelo para que se informen mutuamente. La atención cruzada — el mecanismo del transformer aplicado entre modalidades — es hoy el pegamento de los LLM multimodales (Flamingo, LLaVA, GPT-4V) y prepara el terreno para percepción con sensores (071) y el proyecto integrador (072).

📖 Fundamentos

🧱 Tres estrategias de fusión

- **Fusión temprana (early):** concatenar las características crudas o de bajo nivel de ambas modalidades y entrenar un único modelo sobre el vector conjunto. Máxima capacidad de capturar interacciones finas, pero exige modalidades sincronizadas, sufre con dimensiones muy dispares y se rompe si falta una modalidad.

- **Fusión tardía (late):** cada modalidad tiene su propio modelo hasta la decisión; se combinan las **salidas** (promedio, producto de probabilidades, votación, meta-modelo). Modular y robusta a modalidades faltantes, pero ciega a interacciones — no puede aprender que "este sonido + esta imagen juntos significan otra cosa".
- **Fusión intermedia:** cada modalidad se codifica por separado y las representaciones intermedias se comunican dentro de la red — típicamente con **atención cruzada**. Es el compromiso dominante: interacción rica sin mezclar señales crudas.

🔀 Atención cruzada, paso a paso

En la autoatención, consultas (Q), claves (K) y valores (V) salen de la misma secuencia. En la **atención cruzada**, Q sale de la modalidad A y K, V de la modalidad B:

```
Q = X_A · W_Q        K = X_B · W_K        V = X_B · W_V

Atención(Q, K, V) = softmax( Q · Kᵀ / √d ) · V
```

Lectura: cada elemento de A (p. ej., un token de texto) pregunta "¿qué partes de B (p. ej., parches de la imagen) me son relevantes?" y recibe un resumen ponderado de B. La dirección importa: texto→imagen y imagen→texto son operaciones distintas con parámetros distintos. En Flamingo, capas de atención cruzada insertadas en un LLM congelado dejan que el texto "consulte" las características visuales sin reentrenar el LLM.

📐 Alineación de espacios de representación

Fusionar exige que las representaciones sean comparables:

- **Alineación contrastiva (CLIP, clase 069):** entrenar los codificadores para que pares correspondientes queden cerca. Requiere grandes volúmenes de pares.
- **Proyección aprendida (LLaVA):** una capa lineal o un MLP pequeño proyecta los embeddings visuales al espacio de tokens de un LLM ya entrenado — la imagen entra "disfrazada" de palabras.
- **Precursor clásico:** CCA (análisis de correlación canónica) buscaba proyecciones lineales maximalmente correlacionadas entre dos vistas; útil como referencia histórica y baseline.

También hay que alinear en el **tiempo** (video↔audio: misma escala temporal) y en la **granularidad** (palabra↔parche, frase↔imagen completa).

🧩 Modalidades faltantes y dominancia

Dos problemas prácticos definen el diseño: (1) **ausencia** — en despliegue el micrófono falla o el usuario no envía imagen; la fusión tardía degrada con gracia, la temprana necesita entrenamiento con *dropout de modalidades* para tolerarlo; (2) **dominancia** — si una modalidad es más predictiva o tiene más señal, el modelo puede ignorar la otra por completo y aparentar ser multimodal sin serlo (se diagnostica evaluando con cada modalidad anulada).

🧮 Ejemplo trabajado

Atención cruzada mínima (d = 2). Un token de texto consulta dos parches de imagen:

```
Q = (2, 0)
K₁ = (1, 0)   V₁ = (1, 0)      ← parche 1
K₂ = (0, 1)   V₂ = (0, 1)      ← parche 2

Puntajes:  Q·K₁/√2 = 2/1.414 = 1.414      Q·K₂/√2 = 0

Softmax:   exp(1.414) = 4.11,  exp(0) = 1.00  → pesos (0.80, 0.20)

Salida:    0.80·V₁ + 0.20·V₂ = (0.80, 0.20)
```

El token de texto se lleva un resumen de la imagen dominado por el parche 1 (el que "apunta" en su misma dirección), sin descartar del todo el parche 2. Con temperatura implícita √d mayor (d = 64 real), los mismos productos punto darían pesos más suaves.

Fusión tardía con desacuerdo. El clasificador de audio dice (0.6, 0.4) y el de visión (0.2, 0.8). Promedio: (0.4, 0.6) → clase 2; producto renormalizado: (0.12, 0.32) → (0.27, 0.73) → clase 2 con más margen. El producto castiga más el desacuerdo; el promedio es más conservador. Ninguno aprendió *por qué* discrepan — eso solo lo puede una fusión con interacción.

📊 Propiedades y comparación

Estrategia	Interacciones entre modalidades	¿Tolera modalidad faltante?	Sincronización requerida	Costo
Temprana	Ricas (a nivel de señal)	Mal (requiere dropout de modalidades)	Alta	Un modelo grande
Tardía	Ninguna (solo decisiones)	Bien	Baja	N modelos + combinador
Intermedia (cross-attention)	Ricas (a nivel de representación)	Regular-bien	Media	Capas de atención extra

⚠️ Errores conceptuales frecuentes

1. **"Concatenar vectores ya es fusionar."** Concatenar solo yuxtapone; si el modelo posterior es poco expresivo, nunca modela interacciones entre modalidades. Fusionar es permitir que una modalidad *modifique la lectura* de la otra.
2. **"La fusión tardía capta interacciones si los modelos son buenos."** No puede: combina decisiones ya tomadas. El sarcasmo (texto positivo + tono negativo) es invisible para una fusión tardía por diseño.

3. **"La atención cruzada es simétrica."** texto→imagen e imagen→texto usan Q, K, V distintos y producen resultados distintos; elegir la dirección es una decisión de arquitectura.
4. **"Más modalidades siempre mejoran."** Una modalidad ruidosa o dominante puede degradar el conjunto; hay que medir la contribución de cada una con ablaciones (anular una modalidad y comparar).
5. **"Los espacios se alinean solos."** Sin pares de entrenamiento o proyección aprendida, los embeddings de dos codificadores independientes no son comparables: la similitud entre espacios no alineados no significa nada.

🚀 Del aprendizaje a la operación

Un sistema multimodal real añade: manejo explícito de modalidades faltantes o corruptas (timeouts del micrófono, imágenes ilegibles) con degradación controlada, ablaciones periódicas para detectar dominancia de una modalidad, sincronización temporal medida (no supuesta) entre flujos, presupuesto de cómputo por modalidad, y monitoreo de deriva por canal — la cámara y el micrófono envejecen a ritmos distintos.

🧪 Laboratorio

```
python lab.py
```

El laboratorio llama a `ai_evolution.labs.run_lab("attention")`. Esta decisión evita una implementación divergente por clase: cada una conserva un entrypoint propio, pero los motores didácticos se prueban como una biblioteca común.

🔍 Evidencia esperada

- tipo de laboratorio y semilla;
- entradas o decisiones observables;
- resultado estructurado;
- lista `evidence` con hechos que pueden inspeccionarse;
- lista `limitations` que impide presentar la demo como producción.

📓 Notebooks

- 📓 `notebook.ipynb`: recorrido guiado con la materia resumida.
- ✍️ `notebook_student.ipynb`: ejercicios para resolver.
- ✅ `notebook_solution.ipynb`: solución de referencia explicada.

📝 Evaluación

Criterio	Peso
Comprensión conceptual	25 %
Ejecución reproducible	25 %
Interpretación basada en evidencia	25 %
Riesgos, límites y mejora propuesta	25 %

Consulta assessment.md para preguntas y criterio de aceptación.

⚠️ Errores comunes

Síntoma	Causa probable	Corrección
El código corre, pero no hay conclusión	Se confundió ejecución con aprendizaje	Explica qué demuestra y qué no demuestra
El resultado cambia sin explicación	No se registró semilla o configuración	Conserva semilla, versión y parámetros
Se promete uso real	Se extrapoló desde una demo educativa	Declara entorno, datos, límites y revisión humana
Se copia una métrica aislada	No existe baseline ni costo de error	Añade comparación y criterio de decisión

❓ Preguntas frecuentes

¿Debo usar una API comercial?
No. El núcleo funciona localmente. Las extensiones LIVE se documentan por separado.

¿El laboratorio representa una implementación industrial?
No por sí solo. Enseña el contrato y el patrón; producción exige integración, seguridad, observabilidad, pruebas y operación.

¿Dónde profundizo?
Revisa las especializaciones enlazadas en el README raíz y la ruta siguiente.

🔗 Referencias

- Baltrušaitis, T., Ahuja, C. y Morency, L.-P. (2017). "Multimodal Machine Learning: A Survey and Taxonomy" — arXiv:1705.09406 — uso: fuente primaria del mecanismo estudiado
- Vaswani, A. et al. (2017). "Attention Is All You Need" — arXiv:1706.03762 — uso: fuente primaria del mecanismo estudiado
- Alayrac, J.-B. et al. (2022). "Flamingo: a Visual Language Model for Few-Shot Learning" — arXiv:2204.14198 — uso: fuente primaria del mecanismo estudiado
- Liu, H. et al. (2023). "Visual Instruction Tuning" (LLaVA) — arXiv:2304.08485 — uso: fuente primaria del mecanismo estudiado
- Radford, A. et al. (2021). "Learning Transferable Visual Models From Natural Language Supervision" (CLIP) — arXiv:2103.00020 — uso: fuente primaria del mecanismo estudiado

📜 Papers que fundamentan esta clase

> Bloque generado por `python scripts/link_papers_to_classes.py`. La fuente es `papers/catalog/papers.json`.

Paper	Año	Qué desbloqueó	Miniatura
P18 · Aprender modelos visuales transferibles con supervisión de lenguaje natural	2021	El texto se convierte en la etiqueta: un solo modelo clasifica categorías que nadie anotó, describiéndolas con palabras.	notebook

Cada ficha explica el problema anterior, la matemática mínima, los límites y los errores de atribución más frecuentes. Para leerlas con método: cómo leer un paper de IA · anexos matemáticos.

📚 Bibliografía de apoyo

> Bloque generado por `python scripts/link_sources_to_classes.py`. Cada obra lleva su localizador verificado en `sources/bibliography.json`.

Los papers dicen **de dónde salió** el mecanismo. Estas obras lo **desarrollan** con el espacio que una clase no tiene: teoría completa, demostraciones y ejercicios.

Obra	Edición	Localizador	Papel en esta clase
Jurafsky, Daniel y Martin, James H. — *Speech and Language Processing*	2.ª (la 3.ª circula como borrador abierto sin ISBN) · 2009	ISBN 9780131873216 · web de la obra	obra de referencia de la parte 05 · lenguaje y habla

📝 Evaluación completa

❓ Preguntas

1. Define fusión multimodal y representación conjunta sin usar una marca o framework como definición.
2. Explica la relación entre fusión, cross-attention, modalidades, alineamiento.
3. Ejecuta `lab.py` dos veces con la misma semilla. ¿Qué debe conservarse?
4. Identifica una afirmación permitida y una afirmación exagerada sobre el resultado.
5. Propón una prueba negativa o un caso límite.

🏆 Reto verificable

Amplía el resultado del laboratorio con una clave `student_extension` que incluya:

- el supuesto que estás probando;

- una medición o comprobación;
- la conclusión;
- una limitación.

✅ Criterio de aceptación

- [ ] `lab.py` termina con código 0.
- [ ] El resultado contiene `kind`, `seed`, `evidence` y `limitations`.
- [ ] La extensión no modifica el comportamiento de otras clases.
- [ ] La interpretación referencia datos impresos por el laboratorio.
- [ ] Se declara al menos un riesgo o condición de no uso.

---

071 — Sensores, series y percepción en el borde

Parte: 05 — Lenguaje, visión, audio e IA multimodal
Nivel: avanzado · **Horas estimadas:** 6
Laboratorio: `robotics` · **Estado:** `EXECUTABLE_CORE`

🎯 Propósito

Comprender **sensores, series y percepción en el borde** dentro de la evolución de la inteligencia artificial, implementar un experimento mínimo verificable y distinguir qué parte constituye evidencia frente a una afirmación todavía no comprobada.

📚 Resultados de aprendizaje

Al finalizar podrás:

1. Explicar sensores, series y percepción en el borde usando los conceptos `sensores`, `edge`, `latencia`, `fusión`.
2. Ejecutar el laboratorio con una semilla explícita y revisar su contrato JSON.
3. Identificar al menos un supuesto, una limitación y un riesgo de aplicación.
4. Comparar el enfoque con la etapa anterior de la ruta de aprendizaje.
5. Producir una evidencia reproducible y una conclusión que no exceda los datos.

🧩 Conceptos centrales

`sensores`, `edge`, `latencia`, `fusión`

🗺️ Ubicación en el mapa de la IA

La percepción no termina en cámaras y micrófonos: acelerómetros, giroscopios, sensores de temperatura, presión o corriente producen **series temporales** que también se clasifican y se fusionan. La novedad de esta clase es *dónde* ocurre la inferencia: en el **borde** (edge) — el propio dispositivo — por latencia, energía, costo y privacidad. El TinyML lleva redes neuronales a microcontroladores con kilobytes de RAM, y conecta esta parte con la robótica (parte 06) y con el asistente del proyecto final (072).

📖 Fundamentos

⏱️ Ventanas deslizantes

Un flujo continuo de sensor no se clasifica entero: se corta en **ventanas** de tamaño fijo `w` que avanzan con un salto `h` (hop). Con solapamiento (`h < w`) se generan más ejemplos y no se pierden eventos que caen en el borde de una ventana.

```
n_ventanas = 1 + ⌊(T − w) / h⌋        (T, w, h en muestras)
```

Cada ventana recibe una etiqueta (la actividad dominante) y se convierte en un ejemplo de entrenamiento. Decisiones críticas: `w` debe cubrir el fenómeno (un paso al caminar ~1 s; una caída ~2 s) y `h` fija la latencia de decisión — con hop de 1 s, la respuesta llega como pronto 1 s después.

📈 Features temporales clásicas

Antes (o en lugar) de una red profunda, cada ventana se resume con estadísticos baratos de calcular en un microcontrolador:

- **Dominio del tiempo:** media, desviación estándar, RMS (`√(Σx²/n)`), mínimo/máximo, cruces por cero (ZCR), correlación entre ejes del acelerómetro.
- **Dominio de la frecuencia:** energía por banda de la FFT, frecuencia dominante — caminar concentra energía cerca de 1–2 Hz, correr más arriba.

Un clasificador clásico (árbol, SVM) sobre estas features sigue siendo un baseline duro de batir en reconocimiento de actividad humana (HAR), con una fracción mínima del cómputo.

🔢 Cuantización para el borde

La cuantización mapea pesos y activaciones de `float32` a enteros de 8 bits:

```
x ≈ s · (q − z)      q = clamp( round(x/s) + z , −128, 127 )
```

con `s` (escala) y `z` (punto cero) elegidos por tensor o por canal. En el esquema simétrico, `z = 0` y `s = max|x| / 127`. Beneficios: 4× menos memoria, aritmética entera (más rápida y eficiente en energía, disponible en MCU sin FPU). Costo típico: ~1 punto de exactitud con **cuantización post-entrenamiento**; el **entrenamiento consciente de cuantización (QAT)** simula el redondeo durante el entrenamiento y recupera casi todo.

👆 TinyML: inferencia en microcontroladores

Un MCU típico (Cortex-M4) ofrece ~64–256 kB de RAM, ~1 MB de flash y consumo de miliwatts: el modelo completo (pesos + activaciones + buffers) debe caber ahí. Runtimes como TensorFlow Lite Micro / LiteRT interpretan el modelo cuantizado sin sistema operativo. Técnicas complementarias: *pruning* (podar pesos), *distillation* (entrenar un modelo chico imitando a uno grande) y *duty-cycling* (dormir el sensor y despertar ante un umbral — p. ej., un detector de palabra clave de 20 kB que despierta al modelo grande). A cambio: sin reentrenamiento local, deriva de sensores con el tiempo, y depurar en el dispositivo es difícil.

🔒 Por qué en el borde y no en la nube

Latencia (sin ida y vuelta de red: milisegundos en vez de cientos), energía (radio apagada: transmitir suele costar más que computar), autonomía (funciona sin conectividad) y **privacidad**: el audio o el movimiento crudos nunca salen del dispositivo, solo la decisión ("cayó / no cayó").

🧮 Ejemplo trabajado

Ventanas. Acelerómetro a 50 Hz durante 10 s → T = 500 muestras. Ventana de 2 s (w = 100) con hop de 1 s (h = 50):

```
n = 1 + ⌊(500 − 100) / 50⌋ = 1 + 8 = 9 ventanas
```

Features de una ventana corta. Serie `[2, −2, 2, −2, 2, −2]`:

```
media = 0        RMS = √(Σx²/6) = √(24/6) = 2        cruces por cero = 5
```

Media nula pero RMS alta y ZCR alta: firma de vibración, no de reposo — por eso la media sola no discrimina.

Cuantización int8 simétrica. Pesos en $[-2, 2] \rightarrow s = 2/127 \approx 0.01575$. Para $x = 0.9$: $q = \text{round}(0.9/0.01575) = \text{round}(57.14) = 57$; dequantizado: $57 \cdot 0.01575 \approx 0.898$; error ≈ 0.002. **Memoria:** un modelo de 40 000 parámetros pasa de 160 kB (float32) a 40 kB (int8): entra en un MCU de 64 kB de RAM que antes no podía alojarlo.

📊 Propiedades y comparación

Dónde infiere	Latencia típica	Energía	Privacidad	Tamaño de modelo viable
Nube (GPU)	100–1000 ms (red incluida)	Alta (transmisión + datacenter)	Datos crudos salen del dispositivo	Sin límite práctico
Edge local (móvil/Jetson)	10–100 ms	Media	Datos quedan cerca	Cientos de MB
Microcontrolador (TinyML)	1–50 ms	mW (batería de meses)	Datos nunca salen	10 kB – 1 MB

⚠️ Errores conceptuales frecuentes

1. **"Más frecuencia de muestreo siempre ayuda."** Duplica memoria, cómputo y energía; si el fenómeno vive bajo 10 Hz (movimiento humano), muestrear a 500 Hz solo añade ruido y gasto. La frecuencia se elige por el contenido espectral del fenómeno (Nyquist).
2. **"La cuantización arruina el modelo."** La caída típica post-entrenamiento es ~1 punto, y QAT la reduce más. El error conceptual inverso también existe: hay capas sensibles (primeras/últimas) que a veces conviene dejar en mayor precisión.
3. **"Evalúo separando ventanas al azar."** Con solapamiento, ventanas casi idénticas del mismo gesto caen en train y test: exactitud inflada. La partición correcta es **por sujeto o por sesión**, nunca por ventana.

4. **"La exactitud del paper se traslada al dispositivo."** Cambian el sensor, su posición en el cuerpo, la calibración y la deriva térmica; sin datos del despliegue real, el número del benchmark es una cota optimista.
5. **"TinyML es el mismo modelo pero más chico."** Es otro régimen de diseño: memoria de activaciones, aritmética entera, sin SO, sin reentrenamiento local; la arquitectura se elige por huella de memoria pico, no solo por FLOPs.

🚀 Del aprendizaje a la operación

Un sistema de percepción en el borde añade: calibración por unidad fabricada y compensación de deriva, actualización de modelos por lotes (OTA) con rollback, telemetría agregada que respete la privacidad (contadores, no señal cruda), pruebas de batería en condiciones reales de duty-cycle, y un plan para el caso "el modelo se equivoca y no hay nube que lo corrija": umbrales conservadores y confirmación multi-ventana antes de alertar.

🧪 Laboratorio

```
python lab.py
```

El laboratorio llama a `ai_evolution.labs.run_lab("robotics")`. Esta decisión evita una implementación divergente por clase: cada una conserva un entrypoint propio, pero los motores didácticos se prueban como una biblioteca común.

🔍 Evidencia esperada

- tipo de laboratorio y semilla;
- entradas o decisiones observables;
- resultado estructurado;
- lista `evidence` con hechos que pueden inspeccionarse;
- lista `limitations` que impide presentar la demo como producción.

📓 Notebooks

- 📓 `notebook.ipynb`: recorrido guiado con la materia resumida.
- ✍️ `notebook_student.ipynb`: ejercicios para resolver.
- ✅ `notebook_solution.ipynb`: solución de referencia explicada.

📝 Evaluación

Criterio	Peso
Comprensión conceptual	25 %
Ejecución reproducible	25 %
Interpretación basada en evidencia	25 %
Riesgos, límites y mejora propuesta	25 %

Consulta assessment.md para preguntas y criterio de aceptación.

⚠️ Errores comunes

Síntoma	Causa probable	Corrección
El código corre, pero no hay conclusión	Se confundió ejecución con aprendizaje	Explica qué demuestra y qué no demuestra
El resultado cambia sin explicación	No se registró semilla o configuración	Conserva semilla, versión y parámetros
Se promete uso real	Se extrapoló desde una demo educativa	Declara entorno, datos, límites y revisión humana
Se copia una métrica aislada	No existe baseline ni costo de error	Añade comparación y criterio de decisión

❓ Preguntas frecuentes

¿Debo usar una API comercial?
No. El núcleo funciona localmente. Las extensiones LIVE se documentan por separado.

¿El laboratorio representa una implementación industrial?
No por sí solo. Enseña el contrato y el patrón; producción exige integración, seguridad, observabilidad, pruebas y operación.

¿Dónde profundizo?
Revisa las especializaciones enlazadas en el README raíz y la ruta siguiente.

🔗 Referencias

- Jacob, B. et al. (2017). "Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference" — arXiv:1712.05877 — uso: fuente primaria del mecanismo estudiado
- Banbury, C. et al. (2021). "MLPerf Tiny Benchmark" — arXiv:2106.07597 — uso: fuente primaria del mecanismo estudiado
- Warden, P. y Situnayake, D. (2019). *TinyML: Machine Learning with TensorFlow Lite on Arduino and Ultra-Low-Power Microcontrollers*. O'Reilly. — uso: desarrollo extendido del tema
- Documentación de LiteRT / TensorFlow Lite (cuantización e inferencia en dispositivo) — ai.google.dev/edge/litert — uso: referencia consultada en su fuente original
- UCI Machine Learning Repository. *Human Activity Recognition Using Smartphones* — archive.ics.uci.edu/dataset/240 — uso: referencia consultada en su fuente original

---

📜 Papers que fundamentan esta clase

> Bloque generado por `python scripts/link_papers_to_classes.py`. La fuente es `papers/catalog/papers.json`.

Paper	Año	Qué desbloqueó	Miniatura
P121 · MobileNets: redes convolucionales eficientes para visión en dispositivos móviles	2017	Descompone la convolución en dos pasos y convierte el compromiso entre precisión y coste en dos perillas explícitas que el ingeniero elige.	notebook

Cada ficha explica el problema anterior, la matemática mínima, los límites y los errores de atribución más frecuentes. Para leerlas con método: cómo leer un paper de IA · anexos matemáticos.

📚 Bibliografía de apoyo

> Bloque generado por `python scripts/link_sources_to_classes.py`. Cada obra lleva su localizador verificado en `sources/bibliography.json`.

Los papers dicen **de dónde salió** el mecanismo. Estas obras lo **desarrollan** con el espacio que una clase no tiene: teoría completa, demostraciones y ejercicios.

Obra	Edición	Localizador	Papel en esta clase
Warden, P. y Situnayake, D. — *TinyML: Machine Learning with TensorFlow Lite on Arduino and Ultra-Low-Power Microcontrollers*	2019	*sin localizador verificado*	citada en las referencias de esta clase
Jurafsky, Daniel y Martin, James H. — *Speech and Language Processing*	2.ª (la 3.ª circula como borrador abierto sin ISBN) · 2009	ISBN 9780131873216 · web de la obra	obra de referencia de la parte 05 · lenguaje y habla

📝 Evaluación completa

❓ Preguntas

1. Define sensores, series y percepción en el borde sin usar una marca o framework como definición.
2. Explica la relación entre sensores, edge, latencia, fusión.
3. Ejecuta `lab.py` dos veces con la misma semilla. ¿Qué debe conservarse?
4. Identifica una afirmación permitida y una afirmación exagerada sobre el resultado.
5. Propón una prueba negativa o un caso límite.

🏆 Reto verificable

Amplía el resultado del laboratorio con una clave `student_extension` que incluya:

- el supuesto que estás probando;
- una medición o comprobación;
- la conclusión;
- una limitación.

✅ Criterio de aceptación

- [ ] `lab.py` termina con código 0.
- [ ] El resultado contiene `kind`, `seed`, `evidence` y `limitations`.
- [ ] La extensión no modifica el comportamiento de otras clases.
- [ ] La interpretación referencia datos impresos por el laboratorio.
- [ ] Se declara al menos un riesgo o condición de no uso.

---

072 — Proyecto: asistente multimodal accesible

Parte: 05 — Lenguaje, visión, audio e IA multimodal
Nivel: avanzado · **Horas estimadas:** 10
Laboratorio: `capstone` · **Estado:** `EXECUTABLE_CORE`

🎯 Propósito

Comprender **proyecto: asistente multimodal accesible** dentro de la evolución de la inteligencia artificial, implementar un experimento mínimo verificable y distinguir qué parte constituye evidencia frente a una afirmación todavía no comprobada.

📚 Resultados de aprendizaje

Al finalizar podrás:

1. Explicar proyecto: asistente multimodal accesible usando los conceptos `texto`, `imagen`, `audio`, `accesibilidad`.
2. Ejecutar el laboratorio con una semilla explícita y revisar su contrato JSON.
3. Identificar al menos un supuesto, una limitación y un riesgo de aplicación.
4. Comparar el enfoque con la etapa anterior de la ruta de aprendizaje.
5. Producir una evidencia reproducible y una conclusión que no exceda los datos.

🧩 Conceptos centrales

`texto`, `imagen`, `audio`, `accesibilidad`

🗺️ Ubicación en el mapa de la IA

Esta clase integra toda la parte 05: visión (061-063), lenguaje (064-066), voz (067-068), alineamiento visión-lenguaje (069) y fusión (070) se ensamblan en un asistente que sustituye modalidades — describe imágenes a quien no ve, subtitula audio a quien no oye, lee en voz alta a quien no puede leer. La tecnología asistiva es además un caso ejemplar del "efecto bordillo" (*curb-cut effect*): lo diseñado para discapacidad termina sirviendo a todos, como los subtítulos o el dictado por voz.

📖 Fundamentos

♿ Accesibilidad y WCAG

Las **WCAG** (Web Content Accessibility Guidelines, W3C) son el estándar de referencia. Se organizan en cuatro principios (**POUR**):

- **Perceptible:** la información debe poder percibirse por al menos un sentido disponible — texto alternativo para imágenes (criterio 1.1.1), subtítulos para audio (1.2.2), contraste de texto ≥ 4.5:1 (1.4.3).

- **Operable:** la interfaz debe poder manejarse — todo disponible por teclado (2.1.1), sin límites de tiempo rígidos, sin destellos que provoquen convulsiones.
- **Comprensible:** lenguaje claro, comportamiento predecible, ayuda ante errores.
- **Robusto:** compatible con tecnologías asistivas (lectores de pantalla, ARIA).

Cada criterio tiene nivel **A** (mínimo), **AA** (objetivo legal habitual) o **AAA** (exigente). El **contraste** se calcula sobre la luminancia relativa L de cada color:

```
ratio = (L_claro + 0.05) / (L_oscuro + 0.05)      AA: ≥ 4.5   AAA: ≥ 7
```

🔧 Arquitectura del asistente multimodal

El asistente encadena piezas que ya conoces, cada una con su latencia y su modo de fallo:

```
entrada:  audio → ASR (067)          imagen → VLM descripción (069)      texto directo
núcleo:   fusión + diálogo (070)     con contexto del usuario
salida:   TTS (068)                  texto en pantalla (contraste AA)    subtítulos
```

El **presupuesto de latencia** se reparte: para una interacción conversacional aceptable (~1.5 s extremo a extremo), cada etapa tiene un tope y la más lenta domina. Y cada etapa falla distinto: el ASR degrada con acentos (WER por subgrupo, clase 067), el VLM **alucina detalles** que no están en la imagen, el TTS pronuncia mal lo no normalizado.

🧪 Evaluación con usuarios reales

Las métricas automáticas (WER, similitud) no miden si una persona ciega puede confiar en la descripción. La evaluación asistiva exige:

1. **Métricas por subgrupo:** WER por acento/edad, calidad de descripción por tipo de escena — el promedio esconde a quién falla.
2. **Pruebas con usuarios de tecnología asistiva** en tareas reales, con su propio lector de pantalla y su propio ritmo — el principio "nada sobre nosotros sin nosotros": las personas con discapacidad participan en el diseño, no solo en el test final.
3. **Consentimiento y privacidad:** la cámara y el micrófono de un asistente capturan terceros que no consintieron; procesar en el borde (071) y retener lo mínimo es una decisión de diseño, no un detalle.

⚖️ El riesgo específico: confianza en salidas alucinadas

Para un usuario que **no puede verificar** la salida por otro canal, una descripción inventada no es un error menor: es información falsa entregada con voz segura ("el medicamento vence en 2027" cuando la etiqueta dice 2025). Mitigaciones: expresar incertidumbre en la interfaz, negarse ante baja confianza, ofrecer verificación alternativa (acercar la cámara, pedir segunda foto) y no usar la demo en decisiones médicas, legales o financieras sin revisión humana.

🧮 Ejemplo trabajado

Contraste WCAG. Texto gris sobre fondo blanco: L_blanco = 1.0, L_gris = 0.35.

```
ratio = (1.0 + 0.05) / (0.35 + 0.05) = 1.05 / 0.40 = 2.63   → falla AA (< 4.5)

Con gris más oscuro (L = 0.15): 1.05 / 0.20 = 5.25          → pasa AA, falla AAA
Con casi negro (L = 0.05):      1.05 / 0.10 = 10.5          → pasa AAA
```

Presupuesto de latencia. Objetivo: ≤ 1.5 s de la pregunta hablada a la respuesta hablada.

```
ASR (streaming)            300 ms
Descripción VLM            700 ms
Composición de respuesta   150 ms
TTS (primeras muestras)    250 ms
Red / colas                150 ms
                          ─────────
Total                    1 550 ms   → 50 ms sobre presupuesto: la VLM es el cuello;
                                      recortar ahí (modelo menor, imagen reducida)
                                      rinde más que optimizar el resto.
```

📊 Propiedades y comparación

Método de evaluación	Qué detecta	Qué NO detecta	Costo
Métricas automáticas (WER, similitud)	Regresiones, comparación de modelos	Utilidad real, confianza, contexto	Bajo, continuo
Auditoría WCAG (checklist)	Incumplimientos objetivos (contraste, alt)	Fricción de uso real	Medio, puntual
Pruebas con usuarios asistivos	Barreras reales, confianza, flujo	Cobertura estadística amplia	Alto, imprescindible

⚠️ Errores conceptuales frecuentes

1. **"La accesibilidad se agrega al final."** Ajustar contraste y alt-text sobre un diseño cerrado produce parches; los criterios WCAG condicionan arquitectura (streaming para subtítulos en vivo, estados operables por teclado) y se diseñan desde el inicio.
2. **"Cumplir el checklist = ser usable."** La conformidad AA es necesaria pero no suficiente: un flujo puede pasar la auditoría y seguir siendo inutilizable con un lector de pantalla real. Solo las pruebas con usuarios lo revelan.
3. **"La descripción automática siempre ayuda."** Una alucinación entregada a quien no puede verificarla es peor que un honesto "no puedo leer la etiqueta". La utilidad depende de la calibración de confianza, no solo de la exactitud media.

4. **"Un usuario con discapacidad representa a todos."** Ceguera, baja visión, sordera, movilidad reducida y discapacidad cognitiva imponen requisitos distintos y a veces en tensión; el muestreo de evaluación debe cubrir la diversidad de usuarios objetivo.
5. **"Esto solo beneficia a una minoría."** Efecto bordillo: subtítulos en el metro, manos libres al conducir, dictado por voz — la sustitución de modalidades sirve a cualquiera en situación de discapacidad temporal o contextual.

🚀 Del aprendizaje a la operación

Convertir este proyecto en producto exige: conformidad WCAG AA auditada por terceros, pruebas continuas con usuarios de tecnología asistiva remunerados, calibración y umbrales de rechazo por componente (ASR, VLM, TTS) con métricas por subgrupo, procesamiento local o minimización de datos para cámara/micrófono, y un límite de uso declarado: sin revisión humana no se responde sobre medicación, dinero ni seguridad personal.

🧪 Laboratorio

```
python lab.py
```

El laboratorio llama a `ai_evolution.labs.run_lab("capstone")`. Esta decisión evita una implementación divergente por clase: cada una conserva un entrypoint propio, pero los motores didácticos se prueban como una biblioteca común.

🔍 Evidencia esperada

- tipo de laboratorio y semilla;
- entradas o decisiones observables;
- resultado estructurado;
- lista `evidence` con hechos que pueden inspeccionarse;
- lista `limitations` que impide presentar la demo como producción.

📓 Notebooks

- 📓 `notebook.ipynb`: recorrido guiado con la materia resumida.
- ✍️ `notebook_student.ipynb`: ejercicios para resolver.
- ✅ `notebook_solution.ipynb`: solución de referencia explicada.

📝 Evaluación

Criterio	Peso
Comprensión conceptual	25 %
Ejecución reproducible	25 %
Interpretación basada en evidencia	25 %
Riesgos, límites y mejora propuesta	25 %

Consulta assessment.md para preguntas y criterio de aceptación.

⚠️ Errores comunes

Síntoma	Causa probable	Corrección
El código corre, pero no hay conclusión	Se confundió ejecución con aprendizaje	Explica qué demuestra y qué no demuestra
El resultado cambia sin explicación	No se registró semilla o configuración	Conserva semilla, versión y parámetros
Se promete uso real	Se extrapoló desde una demo educativa	Declara entorno, datos, límites y revisión humana
Se copia una métrica aislada	No existe baseline ni costo de error	Añade comparación y criterio de decisión

❓ Preguntas frecuentes

¿Debo usar una API comercial?
No. El núcleo funciona localmente. Las extensiones LIVE se documentan por separado.

¿El laboratorio representa una implementación industrial?
No por sí solo. Enseña el contrato y el patrón; producción exige integración, seguridad, observabilidad, pruebas y operación.

¿Dónde profundizo?
Revisa las especializaciones enlazadas en el README raíz y la ruta siguiente.

🔗 Referencias

- W3C. *Web Content Accessibility Guidelines (WCAG) 2.2* — w3.org/TR/WCAG22 — uso: marco normativo de referencia
- W3C WAI. *Introduction to Web Accessibility* — w3.org/WAI/fundamentals/accessibility-intro — uso: marco normativo de referencia
- WebAIM. *Contrast Checker* (cálculo de luminancia y ratio WCAG) — webaim.org/resources/contrastchecker — uso: referencia consultada en su fuente original
- Radford, A. et al. (2022). "Robust Speech Recognition via Large-Scale Weak Supervision" (Whisper) — arXiv:2212.04356 — uso: fuente primaria del mecanismo estudiado
- Radford, A. et al. (2021). "Learning Transferable Visual Models From Natural Language Supervision" (CLIP) — arXiv:2103.00020 — uso: fuente primaria del mecanismo estudiado
- Jurafsky, D. y Martin, J. H. *Speech and Language Processing* (3e) — web.stanford.edu/~jurafsky/slp3 — uso: desarrollo extendido del tema

---

📚 Bibliografía de apoyo

> Bloque generado por `python scripts/link_sources_to_classes.py`. Cada obra lleva su localizador verificado en `sources/bibliography.json`.

Los papers dicen **de dónde salió** el mecanismo. Estas obras lo **desarrollan** con el espacio que una clase no tiene: teoría completa, demostraciones y ejercicios.

Obra	Edición	Localizador	Papel en esta clase
Jurafsky, Daniel y Martin, James H. — *Speech and Language Processing*	2.ª (la 3.ª circula como borrador abierto sin ISBN) · 2009	ISBN 9780131873216 · web de la obra	citada en las referencias de esta clase · obra de referencia de la parte 05

Normas y documentación oficial que aplica esta clase: w3.org/TR/WCAG22 · w3.org/WAI/fundamentals/accessibility-intro

📝 Evaluación completa

❓ Preguntas

1. Define proyecto: asistente multimodal accesible sin usar una marca o framework como definición.
2. Explica la relación entre texto, imagen, audio, accesibilidad.
3. Ejecuta `lab.py` dos veces con la misma semilla. ¿Qué debe conservarse?
4. Identifica una afirmación permitida y una afirmación exagerada sobre el resultado.
5. Propón una prueba negativa o un caso límite.

🏆 Reto verificable

Amplía el resultado del laboratorio con una clave `student_extension` que incluya:

- el supuesto que estás probando;
- una medición o comprobación;
- la conclusión;
- una limitación.

✅ Criterio de aceptación

- [ ] `lab.py` termina con código 0.
- [ ] El resultado contiene `kind`, `seed`, `evidence` y `limitations`.
- [ ] La extensión no modifica el comportamiento de otras clases.
- [ ] La interpretación referencia datos impresos por el laboratorio.
- [ ] Se declara al menos un riesgo o condición de no uso.
